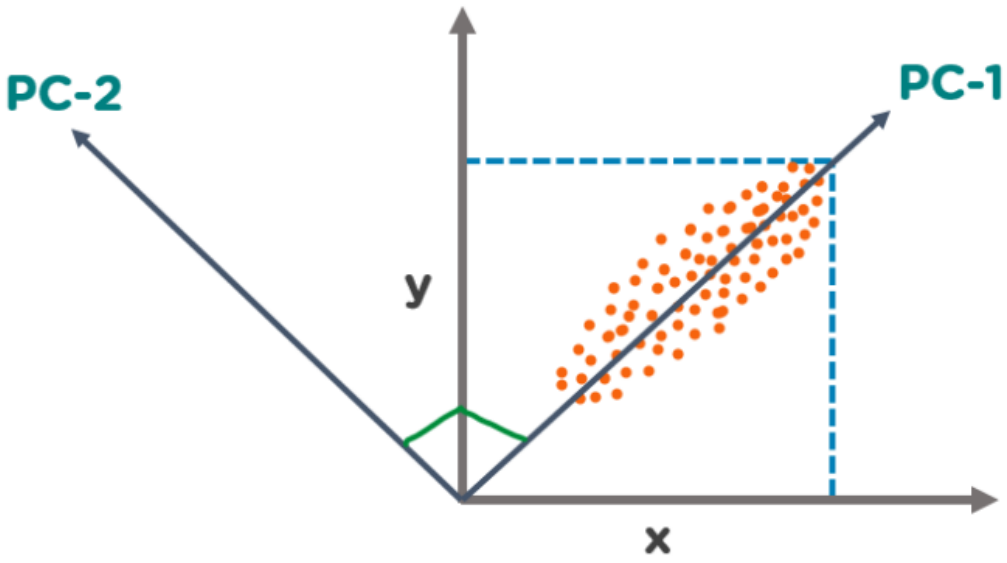
What is Principal Component Analysis (PCA)?

The Principal Component Analysis is a popular unsupervised learning technique for reducing the dimensionality of large data sets. It increases interpretability yet, at the same time, it minimizes information loss. It helps to find the most significant features in a dataset and makes the data easy for plotting in 2D and 3D. PCA helps in finding a sequence of linear combinations of variables.



In the above figure, we have several points plotted on a 2-D plane. There are two principal components. PC1 is the primary principal component that explains the maximum variance in the data. PC2 is another principal component that is orthogonal to PC1.

What is a Principal Component?

The Principal Components are a straight line that captures most of the variance of the data. They have a direction and magnitude. Principal components are orthogonal projections (perpendicular) of data onto lower-dimensional space.

Dimensionality

The term "dimensionality" describes the quantity of features or variables used in the research. It can be difficult to visualize and interpret the relationships between variables when dealing with high-dimensional data, such as datasets with numerous variables. While reducing the number of variables in the dataset, dimensionality reduction methods like PCA are used to preserve the most crucial data. The original variables are converted into a new set of variables called principal components, which are linear combinations of the original variables, by PCA in order to accomplish this. The dataset's reduced dimensionality depends on how many principal components are used in the study. The objective of PCA is to select fewer principal components that account for the data's most important variation. PCA can help to streamline data analysis, enhance visualization, and make it simpler to spot trends and relationships between factors by reducing the dimensionality of the dataset.

The mathematical representation of dimensionality reduction in the context of PCA is as follows:

Given a dataset with n observations and p variables represented by the n x p data matrix X, the goal of PCA is to transform the original variables into a new set of k variables called principal components that capture the most significant variation in the data. The principal components are defined as linear combinations of the original variables given by:

PC_1 = a_11 * x_1 + a_12 * x_2 + ... + a_1p * x_p

PC_2 = a_21 * x_1 + a_22 * x_2 + ... + a_2p * x_p

...

PC_k = a_k1 * x_1 + a_k2 * x_2 + ... + a_kp * x_p

where a_ij is the loading or weight of variable x_j on principal component PC_i, and x_j is the jth variable in the data matrix X. The principal components are ordered such that the first component PC_1 captures the most significant variation in the data, the second component PC_2 captures the second most significant variation, and so on. The number of principal components used in the analysis, k, determines the reduced dimensionality of the dataset.

Correlation

A statistical measure known as correlation expresses the direction and strength of the linear connection between two variables. The covariance matrix, a square matrix that displays the pairwise correlations between all pairs of variables in the dataset, is calculated in the setting of PCA using correlation. The covariance matrix's diagonal elements stand for each variable's variance, while the off-diagonal elements indicate the covariances between different pairs of variables. The strength and direction of the linear connection between two variables can be determined using the correlation coefficient, a standardized measure of correlation with a range of -1 to 1.

A correlation coefficient of 0 denotes no linear connection between the two variables, while correlation coefficients of 1 and -1 denote the perfect positive and negative correlations, respectively. The principal components in PCA are linear combinations of the initial variables that maximize the variance explained by the data. Principal components are calculated using the correlation matrix.

In the framework of PCA, correlation is mathematically represented as follows:

The correlation matrix C is a nxn symmetric matrix with the following components given a dataset with n variables (x1, x2,..., xn):

Cij = (sd(xi) * sd(xj)) / cov(xi, xj)

where sd(x i) is the standard deviation of variable x i and sd(x j) is the standard deviation of variable x j, and cov(x i, x j) is the correlation between variables x i and x j.

The correlation matrix C can also be written as follows in matrix notation:

C = X^T X / (n-1) (n-1)

Orthogonal

The term "orthogonality" alludes to the principal components' construction as being orthogonal to one another in the context of the PCA algorithm. This indicates that there is no redundant information among the main components and that they are not correlated with one another.

Orthogonality in PCA is mathematically expressed as follows: each principal component is built to maximize the variance explained by it while adhering to the requirement that it be orthogonal to all other principal components. The principal components are computed as linear combinations of the original variables. Thus, each principal component is guaranteed to capture a unique and non-redundant part of the variation in the data.

The orthogonality constraint is expressed as:

a_i1 * a_j1 + a_i2 * a_j2 + ... + a_ip * a_jp = 0

for all i and j such that $i \neq j$. This means that the dot product between any two loading vectors for different principal components is zero, indicating that the principal components are orthogonal to each other.

Eigen Vectors

The main components of the data are calculated using the eigenvectors. The ways in which the data vary most are represented by the eigenvectors of the data's covariance matrix. The new coordinate system in which the data is represented is then defined using these coordinates.

The covariance matrix C in mathematics is represented by the letters v 1, v 2,..., v p, and the associated eigenvalues are represented by _1, _2,..., _p. The eigenvectors are calculated in such a way that the equation shown below holds:

C v_i = λ_i v_i

This means that the eigenvector v_i produces the associated eigenvalue λ_i as a scalar multiple of itself when multiplied by the covariance matrix C.

Covariance Matrix

The covariance matrix is crucial to the PCA algorithm's computation of the data's main components. The pairwise covariances between the factors in the data are measured by the covariance matrix, which is a p x p matrix.

The correlation matrix C is defined as follows given a data matrix X of n observations of p variables:

C = (1/n) * X^T X

where X^T represents X's transposition. The covariances between the variables are represented by the off-diagonal elements of C, whereas the variances of the variables are represented by the diagonal elements of C.

Steps for PCA Algorithm

1. Standardize the data: PCA requires standardized data, so the first step is to standardize the data to ensure that all variables have a mean of 0 and a standard deviation of 1.
2. Calculate the covariance matrix: The next step is to calculate the covariance matrix of the standardized data. This matrix shows how each variable is related to every other variable in the dataset.
3. Calculate the eigenvectors and eigenvalues: The eigenvectors and eigenvalues of the covariance matrix are then calculated. The eigenvectors represent the directions in which the data varies the most, while the eigenvalues represent the amount of variation along each eigenvector.
4. Choose the principal components: The principal components are the eigenvectors with the highest eigenvalues. These components represent the directions in which the data varies the most and are used to transform the original data into a lower-dimensional space.
5. Transform the data: The final step is to transform the original data into the lower-dimensional space defined by the principal components.

Applications of PCA in Machine Learning

- PCA is used to visualize multidimensional data.
- It is used to reduce the number of dimensions in healthcare data.
- PCA can help resize an image.
- It can be used in finance to analyze stock data and forecast returns.
- PCA helps to find patterns in the high-dimensional datasets

Advantages of PCA

1. Dimensionality reduction: By determining the most crucial features or components, PCA reduces the dimensionality of the data, which is one of its primary benefits. This can be helpful when the initial data contains a lot of variables and is therefore challenging to visualize or analyze.
2. Feature Extraction: PCA can also be used to derive new features or elements from the original data that might be more insightful or understandable than the original features. This is particularly helpful when the initial features are correlated or noisy.
3. Data visualization: By projecting the data onto the first few principal components, PCA can be used to visualize high-dimensional data in two or three dimensions. This can aid in locating data patterns or clusters that may not have been visible in the initial high-dimensional space.
4. Noise Reduction: By locating the underlying signal or pattern in the data, PCA can also be used to lessen the impacts of noise or measurement errors in the data.
5. Multicollinearity: When two or more variables are strongly correlated, there is multicollinearity in the data, which PCA can handle. PCA can lessen the impacts of multicollinearity on the analysis by identifying the most crucial features or components.

Disadvantages of PCA

1. Interpretability: Although principal component analysis (PCA) is effective at reducing the dimensionality of data and spotting patterns, the resulting principal components are not always simple to understand or describe in terms of the original features.
2. Information loss: PCA involves choosing a subset of the most crucial features or components in order to reduce the dimensionality of the data. While this can be helpful for streamlining the data and lowering noise, if crucial features are not included in the components chosen, information loss may also result.
3. Outliers: Because PCA is susceptible to anomalies in the data, the resulting principal components may be significantly impacted. The covariance matrix can be distorted by outliers, which can make it harder to identify the most crucial characteristics.

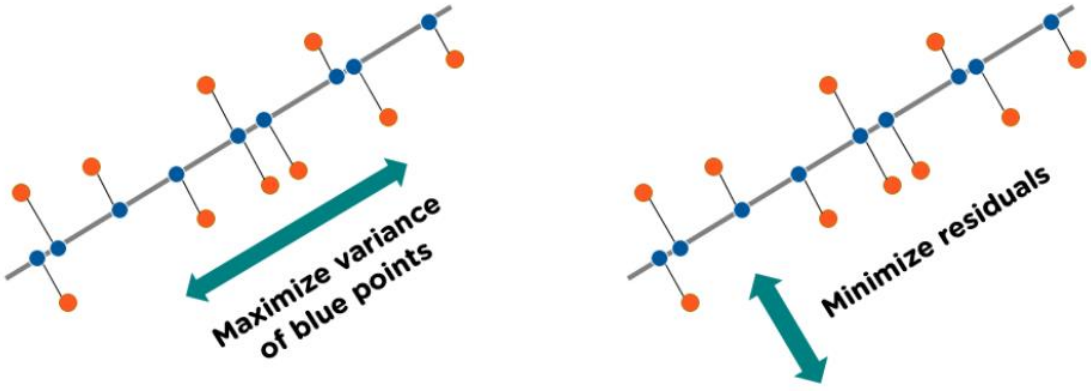
   Scaling: PCA makes the assumption that the data is scaled and centralized, which can be a drawback in some circumstances. The resulting principal components might not correctly depict the underlying patterns in the data if the data is not scaled properly.
5. Computing complexity: For big datasets, it may be costly to compute the eigenvectors and eigenvalues of the covariance matrix. This may restrict PCA's ability to scale and render it useless for some uses.

Uses of PCA

PCA is a widely used technique in data analysis and has a variety of applications, including:

1. Data compression: PCA can be used to reduce the dimensionality of high-dimensional datasets, making them easier to store and analyze.
2. Feature extraction: PCA can be used to identify the most important features in a dataset, which can be used to build predictive models.
3. Visualization: PCA can be used to visualize high-dimensional data in two or three dimensions, making it easier to understand and interpret.
4. Data pre-processing: PCA can be used as a pre-processing step for other machine learning algorithms, such as clustering and classification.

How Does Principal Component Analysis Work?



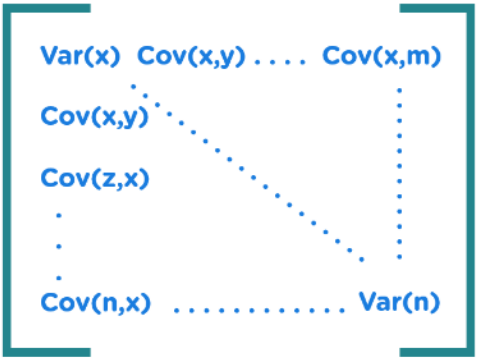
1. Normalize the Data

Standardize the data before performing PCA. This will ensure that each feature has a mean = 0 and variance = 1.

$$Z = \frac{x - \mu}{\sigma}$$

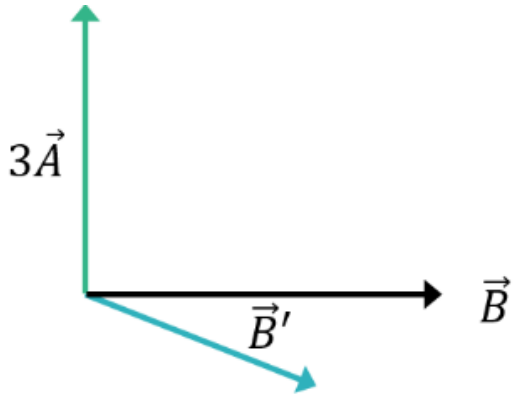
2. Build the Covariance Matrix

Construct a square matrix to express the correlation between two or more features in a multidimensional dataset.



3. Find the Eigenvectors and Eigenvalues

Calculate the eigenvectors/unit vectors and eigenvalues. Eigenvalues are scalars by which we multiply the eigenvector of the covariance matrix.



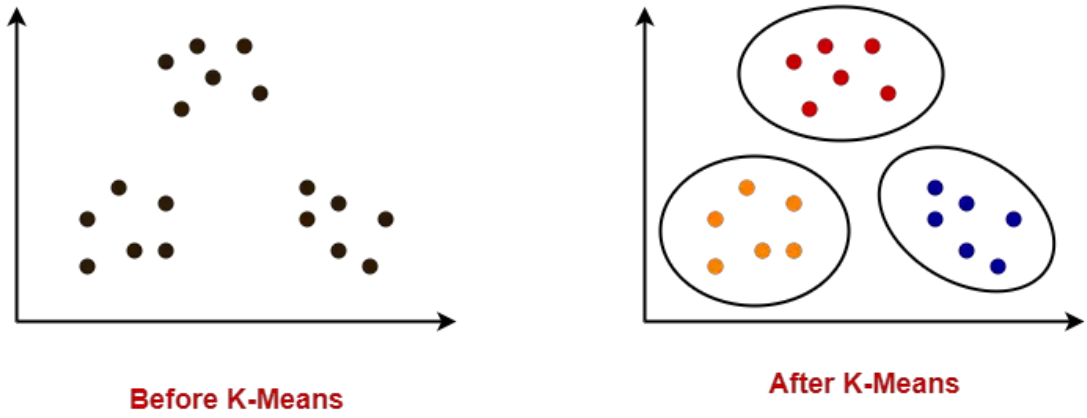
4. Sort the Eigenvectors in Highest to Lowest Order and Select the Number of Principal Components.

K-means Clustering

What is K-means Clustering?

Unsupervised Machine Learning is the process of teaching a computer to use unlabeled, unclassified data and enabling the algorithm to operate on that data without supervision. Without any previous data training, the machine's job in this case is to organize unsorted data according to parallels, patterns, and variations.

- K-Means clustering is an unsupervised iterative clustering technique.
- It partitions the given data set into k predefined distinct clusters.
- A cluster is defined as a collection of data points exhibiting certain similarities.



It partitions the data set such that-

- Each data point belongs to a cluster with the nearest mean.
- Data points belonging to one cluster have high degree of similarity.
- Data points belonging to different clusters have high degree of dissimilarity.

What is the objective of k-means clustering?

The goal of clustering is to divide the population or set of data points into a number of groups so that the data points within each group are more comparable to one another and different from the data points within the other groups. It is essentially a grouping of things based on how similar and different they are to one another.

How k-means clustering works?

We are given a data set of items, with certain features, and values for these features (like a vector). The task is to categorize those items into groups. To achieve this, we will use the K-means algorithm, an unsupervised learning algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.

(It will help if you think of items as points in an n-dimensional space). The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity, we will use the Euclidean distance as a measurement.

The algorithm works as follows:

First, we randomly initialize k points, called means or cluster centroids.

We categorize each item to its closest mean, and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.

We repeat the process for a given number of iterations and at the end, we have our clusters.

The "points" mentioned above are called means because they are the mean values of the items categorized in them. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set (if for a feature x, the items have values in [0,3], we will initialize the means with values for x at [0,3]).

K-Means Clustering Algorithm involves the following steps-

Step-01:

Choose the number of clusters K.

Step-02:

Randomly select any K data points as cluster centers.
Select cluster centers in such a way that they are as farther as possible from each other.

Step-03:

Calculate the distance between each data point and each cluster center.
The distance may be calculated either by using given distance function or by using euclidean distance formula.

Step-04:

Assign each data point to some cluster.
A data point is assigned to that cluster whose center is nearest to that data point.

Step-05:

Re-compute the center of newly formed clusters.

The center of a cluster is computed by taking mean of all the data points contained in that cluster.

Step-06:

Keep repeating the procedure from Step-03 to Step-05 until any of the following stopping criteria is met-

- Center of newly formed clusters do not change
- Data points remain present in the same cluster
- Maximum number of iterations are reached

Advantages-

K-Means Clustering Algorithm offers the following advantages-

1. It is relatively efficient with time complexity O(nkt) where-

   n = number of instances
   k = number of clusters
   t = number of iterations

2. It often terminates at local optimum. Techniques such as Simulated Annealing or **Genetic Algorithms** may be used to find the global optimum.

Disadvantages-

K-Means Clustering Algorithm has the following disadvantages-

1. It requires to specify the number of clusters (k) in advance.
2. It can not handle noisy data and outliers.
3. It is not suitable to identify clusters with non-convex shapes.

Pseudocode of Algorithm

The above algorithm in pseudocode is as follows:

Initialize k means with random values

--> For a given number of iterations:

--> Iterate through items:

--> Find the mean closest to the item by calculating the euclidean distance of the item with each of the means

--> Assign item to mean

--> Update mean by shifting it to the average of the items in that cluster

PRACTICE PROBLEMS BASED ON K-MEANS CLUSTERING ALGORITHM-

Problem-01:

Cluster the following eight points (with (x, y) representing locations) into three clusters:

A1(2, 10), A2(2, 5), A3(8, 4), A4(5, 8), A5(7, 5), A6(6, 4), A7(1, 2), A8(4, 9)

Initial cluster centers are: A1(2, 10), A4(5, 8) and A7(1, 2).

The distance function between two points a = (x1, y1) and b = (x2, y2) is defined as-

$$P(a, b) = |x2 – x1| + |y2 – y1|$$

Use K-Means Algorithm to find the three cluster centers after the second iteration.

Solution-

We follow the above discussed K-Means Clustering Algorithm-

Iteration-01:

- We calculate the distance of each point from each of the center of the three clusters.
- The distance is calculated by using the given distance function.

The following illustration shows the calculation of distance between point A1(2, 10) and each of the center of the three clusters-

Calculating Distance Between A1(2, 10) and C1(2, 10)-

P(A1, C1)

= |x2 – x1| + |y2 – y1|

= |2 – 2| + |10 – 10|

= 0

Calculating Distance Between A1(2, 10) and C2(5, 8)-

P(A1, C2)

= |x2 – x1| + |y2 – y1|

= |5 – 2| + |8 – 10|

= 3 + 2

= 5

Calculating Distance Between A1(2, 10) and C3(1, 2)-

P(A1, C3)

= |x2 – x1| + |y2 – y1|

= |1 – 2| + |2 – 10|

= 1 + 8

= 9

In the similar manner, we calculate the distance of other points from each of the center of the three clusters.

Next,

- We draw a table showing all the results.
- Using the table, we decide which point belongs to which cluster.
- The given point belongs to that cluster whose center is nearest to it.

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (5, 8) of Cluster-02	Distance from center (1, 2) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	5	9	C1
A2(2, 5)	5	6	4	C3
A3(8, 4)	12	7	9	C2

A4(5, 8)	5	0	10	C2
A5(7, 5)	10	5	9	C2
A6(6, 4)	10	5	7	C2
A7(1, 2)	9	10	0	C3
A8(4, 9)	3	2	10	C2

From here, New clusters are-

Cluster-01:

First cluster contains points-

- A1(2, 10)

Cluster-02:

Second cluster contains points-

- A3(8, 4)
- A4(5, 8)
- A5(7, 5)
- A6(6, 4)
- A8(4, 9)

Cluster-03:

Third cluster contains points-

- A2(2, 5)
- A7(1, 2)

Now,

- We re-compute the new cluster clusters.
- The new cluster center is computed by taking mean of all the points contained in that cluster.

For Cluster-01:

We have only one point A1(2, 10) in Cluster-01.

- So, cluster center remains the same.

For Cluster-02:

Center of Cluster-02

= ((8 + 5 + 7 + 6 + 4)/5, (4 + 8 + 5 + 4 + 9)/5)

= (6, 6)

For Cluster-03:

Center of Cluster-03

= ((2 + 1)/2, (5 + 2)/2)

= (1.5, 3.5)

This is completion of Iteration-01.

Iteration-02:

- We calculate the distance of each point from each of the center of the three clusters.
- The distance is calculated by using the given distance function.

The following illustration shows the calculation of distance between point A1(2, 10) and each of the center of the three clusters-

Calculating Distance Between A1(2, 10) and C1(2, 10)-

P(A1, C1)

= |x2 – x1| + |y2 – y1|

= |2 – 2| + |10 – 10|

= 0

Calculating Distance Between A1(2, 10) and C2(6, 6)-

P(A1, C2)

= |x2 – x1| + |y2 – y1|

= |6 – 2| + |6 – 10|

= 4 + 4

= 8

Calculating Distance Between A1(2, 10) and C3(1.5, 3.5)-

P(A1, C3)

= |x2 – x1| + |y2 – y1|

= |1.5 – 2| + |3.5 – 10|

= 0.5 + 6.5

= 7

In the similar manner, we calculate the distance of other points from each of the center of the three clusters.

Next,

- We draw a table showing all the results.
- Using the table, we decide which point belongs to which cluster.
- The given point belongs to that cluster whose center is nearest to it.

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (6, 6) of Cluster-02	Distance from center (1.5, 3.5) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	8	7	C1
A2(2, 5)	5	5	2	C3
A3(8, 4)	12	4	7	C2
A4(5, 8)	5	3	8	C2
A5(7, 5)	10	2	7	C2
A6(6, 4)	10	2	5	C2
A7(1, 2)	9	9	2	C3
A8(4, 9)	3	5	8	C1

From here, New clusters are-

Cluster-01:

First cluster contains points-

- A1(2, 10)
- A8(4, 9)

Cluster-02:

Second cluster contains points-

- A3(8, 4)

- A4(5, 8)
- A5(7, 5)
- A6(6, 4)

Cluster-03:

Third cluster contains points-

- A2(2, 5)
- A7(1, 2)

Now,

- We re-compute the new cluster clusters.
- The new cluster center is computed by taking mean of all the points contained in that cluster.

For Cluster-01:

Center of Cluster-01

= ((2 + 4)/2, (10 + 9)/2)

= (3, 9.5)

For Cluster-02:

Center of Cluster-02

= ((8 + 5 + 7 + 6)/4, (4 + 8 + 5 + 4)/4)

= (6.5, 5.25)

For Cluster-03:

Center of Cluster-03

= ((2 + 1)/2, (5 + 2)/2)

= (1.5, 3.5)

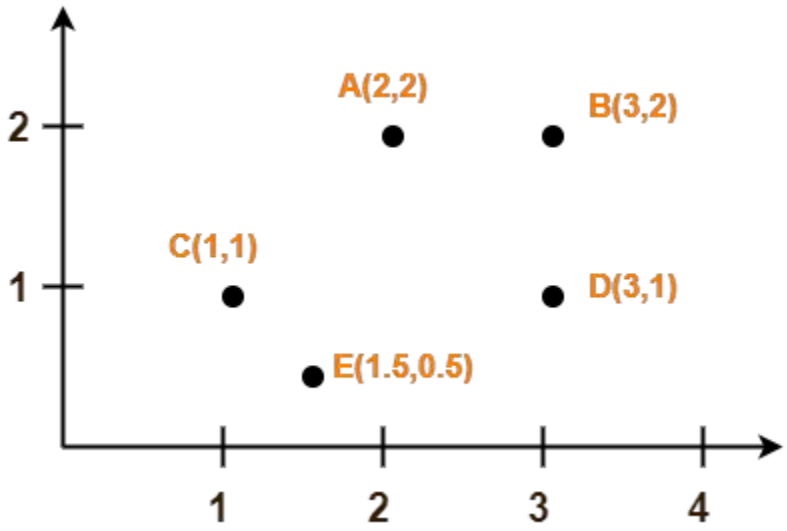
This is completion of Iteration-02.

After second iteration, the center of the three clusters are-

- C1(3, 9.5)
- C2(6.5, 5.25)
- C3(1.5, 3.5)

Problem-02:

Use K-Means Algorithm to create two clusters-



Solution-

We follow the above discussed K-Means Clustering Algorithm.

Assume A(2, 2) and C(1, 1) are centers of the two clusters.

Iteration-01:

- We calculate the distance of each point from each of the center of the two clusters.
- The distance is calculated by using the euclidean distance formula.

The following illustration shows the calculation of distance between point A(2, 2) and each of the center of the two clusters-

Calculating Distance Between A(2, 2) and C1(2, 2)-

P(A, C1)

$= \text{sqrt} [(x2 – x1)^2 + (y2 – y1)^2]$

$= \text{sqrt} [(2 – 2)^2 + (2 – 2)^2]$

$= \text{sqrt} [0 + 0]$

= 0

Calculating Distance Between A(2, 2) and C2(1, 1)-

P(A, C2)

$= \text{sqrt} [(x2 – x1)^2 + (y2 – y1)^2]$

$= \text{sqrt} [(1 – 2)^2 + (1 – 2)^2]$

$= \text{sqrt} [1 + 1]$

$= \text{sqrt} [2]$

$= 1.41$

In the similar manner, we calculate the distance of other points from each of the center of the two clusters.

Next,

- We draw a table showing all the results.
- Using the table, we decide which point belongs to which cluster.
- The given point belongs to that cluster whose center is nearest to it.

Given Points	Distance from center (2, 2) of Cluster-01	Distance from center (1, 1) of Cluster-02	Point belongs to Cluster
A(2, 2)	0	1.41	C1
B(3, 2)	1	2.24	C1
C(1, 1)	1.41	0	C2
D(3, 1)	1.41	2	C1
E(1.5, 0.5)	1.58	0.71	C2

From here, New clusters are-

Cluster-01:

First cluster contains points-

- A(2, 2)

- B(3, 2)
- E(1.5, 0.5)
- D(3, 1)

Cluster-02:

Second cluster contains points-

- C(1, 1)
- E(1.5, 0.5)

Now,

- We re-compute the new cluster clusters.
- The new cluster center is computed by taking mean of all the points contained in that cluster.

For Cluster-01:

Center of Cluster-01

= ((2 + 3 + 3)/3, (2 + 2 + 1)/3)

= (2.67, 1.67)

For Cluster-02:

Center of Cluster-02

= ((1 + 1.5)/2, (1 + 0.5)/2)

= (1.25, 0.75)

This is completion of Iteration-01.

Next, we go to iteration-02, iteration-03 and so on until the centers do not change anymore.

Part I

Ensemble Learning

- Machine learning models are designed to generalize from training data to new, unseen data. However, there is a fundamental trade-off between the bias and variance of a model that can affect its ability to generalize.
- Bias of a model is the error in predicting the training dataset, while variance is the change in model with respect to change in training dataset.
- A model with high bias tends to underfit the training data, i.e. the model is too simple to capture the underlying patterns in the data. In contrast, a model with high variance is prone to memorize the noise in the training data rather than the underlying patterns. In other words, it overfit the training data.
- Therefore, we look for a model which is both low bias and low variance, however it is difficult to construct a single model that simultaneously achieves both of them.

Why is it difficult to construct a low-bias low-variance machine?

The bias-variance trade-off can be mathematically demonstrated using the concept of the mean squared error (MSE) of a machine learning model. The MSE measures the average squared difference between the predicted values and the true values of the target variable in the training data. It can be decomposed into three components: the squared bias, the variance, and the irreducible error:

$$MSE = Bias^2 + Variance + Irreducible\ error\ ^1$$

For a single model, once the minima of MSE is reached, it is impossible to lower both bias and variance simultaneously. However, ensemble learning can be used to tune the bias-variance trade-off by combining models with different levels of bias and variance. Subsequently, combining multiple models can achieve better performance than any individual model.

[1] Irreducible Error represents the minimum achievable error of any model on the given data, due to the inherent noise or variability in the data. The proof of the equation is as follows,

$$\begin{aligned} E[(Y - \hat{Y})^2] &= E[(Y - E[\hat{Y}] + E[\hat{Y}] - \hat{Y})^2] \\ &= E[(Y - E[\hat{Y}])^2] + E[(E[\hat{Y}] - \hat{Y})^2] + 2E[(Y - E[\hat{Y}])(E[\hat{Y}] - \hat{Y})] \\ &= Bias^2 + Variance + Irreducible\ error \end{aligned}$$

- In machine learning, ensemble learning methods are techniques used to combine multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone.
- It works on the hypothesis that certain models do well in one aspect of the data, while others do well in modeling another. The idea is to leverage the strengths of different models and reduce their individual weaknesses.
- There are different types of ensemble learning, each with its own approach to combining models. The most common types are bagging, boosting, and stacking. In bagging, multiple models are trained on different subsets of the training data, which reduces the variance by averaging the predictions of the individual models. Boosting, on the other hand, focuses on reducing bias by sequentially training models on misclassified instances. Stacking combines multiple models by training a meta-model on their predictions. The meta-model learns to combine the strengths of the individual models and generate a final prediction. We'll discuss Bagging and Boosting in detail in later slides.

The Netflix Prize: A fun Example

Ensemble learning has been successfully applied in various real-world scenarios, one such example is the Netflix Prize Challenge.

- In 2006, Netflix launched a competition to improve their recommendation algorithm, which recommends movies and TV shows to users based on their viewing history.
- The goal was to reduce the root mean square error (RMSE) of their existing algorithm by 10%. The competition attracted more than 40,000 teams from around the world, and the winning team, BellKor's Pragmatic Chaos, used an ensemble of multiple machine learning models to achieve a 10.06% improvement in RMSE.
- The winning solution was a blend of 107 different models, each using different combinations of features and algorithms. By combining the predictions of these individual models, the ensemble was able to achieve a much lower RMSE than any individual model.

Part II

Bagging and Boosting

Bagging

Definition and Idea

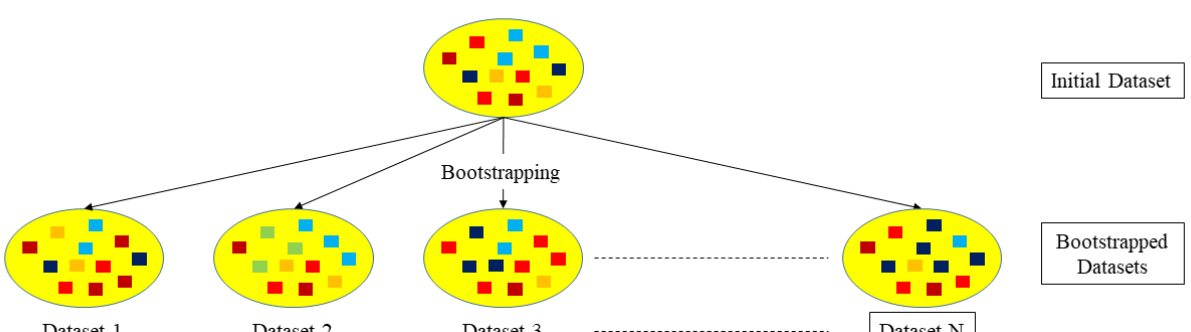
Bagging stands for Bootstrap Aggregating, which is a technique used in ensemble learning to reduce the variance of machine learning models. The idea behind bagging is to train multiple models on different subsets of the training data, and then combine their predictions to make the final prediction.

- In bagging, we randomly select a subset of the training data with replacement (bootstrap) to create a new training set for each model. This means that some data points may be present multiple times in a single subset, while others may be left out entirely.
- By training models on these different subsets, we can reduce the variance of the overall prediction, since each model is making its predictions based on a slightly different set of data. Bagging is particularly useful in applications where the individual models are prone to overfitting the training data, as bagging can help to reduce this overfitting by creating more diverse models.
- It is commonly used in applications such as classification, regression, and time series forecasting. We will discuss the detailed algorithm of Vanilla Bagging (and Random Forest, which is a stretch of this algorithm) in the next slides.

Vanilla Bagging

Vanilla Bagging is widely used in real-world applications including credit risk assessment, fraud detection, stock price prediction etc.

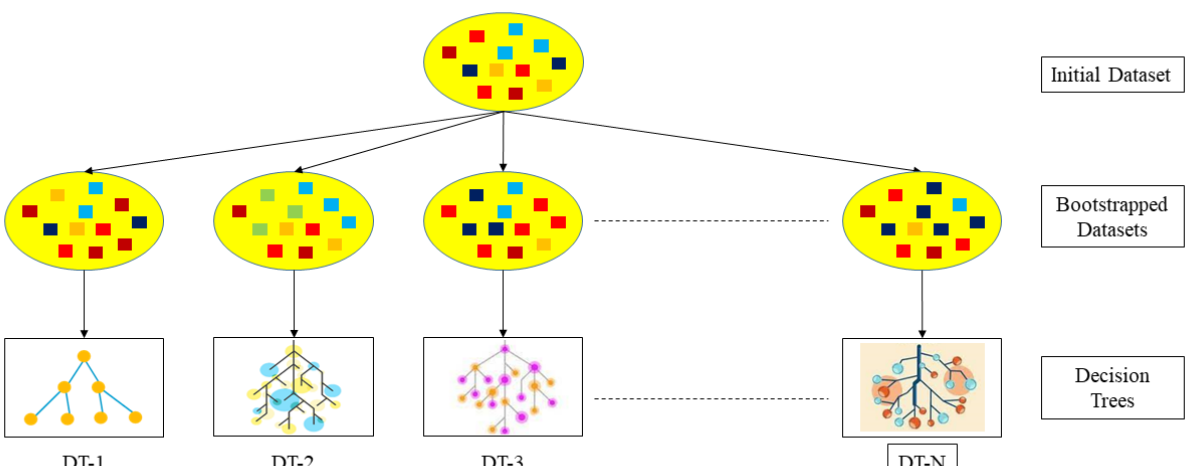
- **Algorithm**
  - In Vanilla Bagging, we use bootstrap method (randomly choosing an element with replacement) to make subset datasets of equal length as the initial dataset.
  - Even though the number of elements in subsets are equal to the number of elements in the original dataset, they are not the same dataset as the draw has been done with replacements.
  - **The number of subsets, N, is a hyperparameter.**



Vanilla Bagging Algorithm

Continued

- After bootstrapping, a separate decision tree is built on each subset of data.
- It is not mandatory to use decision trees, Vanilla Bagging can be used with pretty much any algorithm that can handle multiple inputs, such as decision trees, neural networks, and support vector machines.



Vanilla Bagging Algorithm

Continued

- Each of the models in each subset predicts its own result.

- **Regression Problem:**
  - In regression problems, the final output is calculated as the average of all the predicted results from each tree (or other model).

- **Classification Problem:**
  - For binary classification problems, each model from each subset predicts either 'yes' or 'no'. The final output is determined by aggregating the predictions of all models, with a majority vote.
  - For general cases, the mode of all models is taken as the final output.
  - However, complications may arise in case of a tie. These cases can be handled with domain knowledge, classification with a confidence level or other complicated methods.

Vanilla Bagging Algorithm

Continued

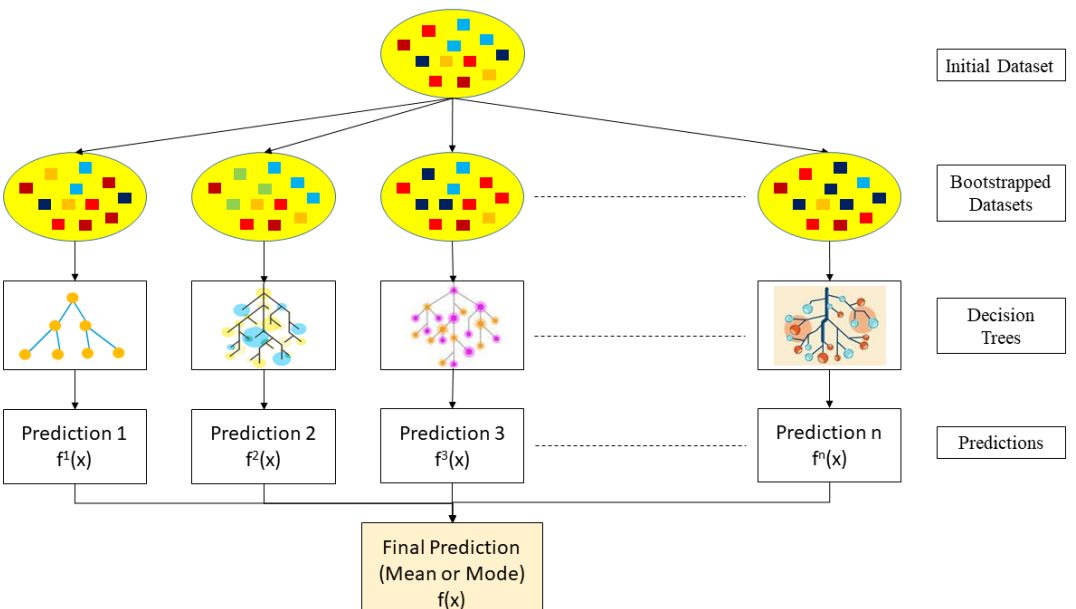


Figure. Algorithm for Vanilla Bagging

Random Forest

- Random Forest is a popular ensemble learning algorithm, which is an extension of the vanilla bagging algorithm.
- The first algorithm for random decision forests was created in 1995 by Tin Kam Ho. In this algorithm, he introduced the idea of random feature selection for a high cardinality of feature space, which is the key difference between vanilla bagging and random forest.
- The algorithm for random forests is similar to that of bagging methods. However, in random forests, a subset of pre-decided length is formed from original feature space for each of the bootstrapped dataset.
- The feature subset is chosen randomly without replacements. **The length of the feature subsets, ξ_f, is a hyperparameter.**
- A decision tree is formed for each dataset and corresponding feature space, leading to a prediction from each. Final prediction is made following the same rules as of bagging, i.e. taking mean for regression and mode for classification.

Random Forest Algorithm

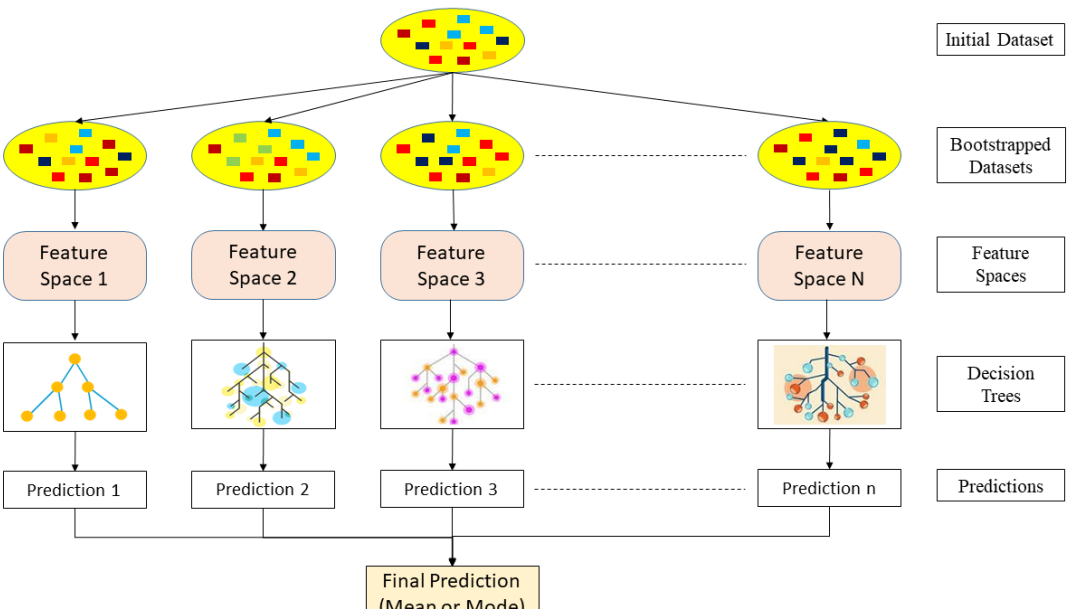


Figure. Algorithm for Random Forest

Random Forest: Key Benefits and Challenges

- **Benefits:**
  - **Reduced risk of overfitting:** Decision trees are prone to overfitting because they try to fit all the samples in the training data too closely. Random Forests with a large number of decision trees reduce the risk of overfitting because the averaging of independent trees lowers the overall variance and prediction error .
  - **High accuracy:** Random Forest can handle both regression and classification problems with high accuracy, making it a popular method among data scientists.
  - **Robustness:** Random Forest is robust to noise and missing values, which makes it a good choice for data with missing or incomplete values.
- **Challenges:**
  - **Time and Resource Demanding:** As Random Forest computes a separate decision tree with different feature space for each subset of data, it takes a lot more time and computation power compared to a simple decision tree.
  - **Interpretability:** Even though Random Forest provides a measure of feature importance, prediction of a single decision tree is easier to interpret since it shows the path of the decision-making process for that specific sample.

Random Forest: Applications

Random Forest, in general, is a good choice when we want a high-performing model with low variance and low bias. It is particularly useful when we have a large number of strongly correlated features, as the feature subsampling helps to decorrelate the models. Although, in instances, when we don't have a large sample-space or feature-space, or we need to find co-dependencies or strong interpretation, it is more useful to use a simpler algorithm such as decision tree or support vector machine. Nevertheless, Random Forest is one of the most powerful machine learning algorithms we have and it's been used in several complicated real life problems.

- Random Forest is used in the **banking and finance** industry for tasks such as credit risk analysis, fraud detection, and loan approval processes.
- In **e-commerce**, it is used for tasks such as customer segmentation, personalized product recommendations, and fraud detection.
- Random Forest is used in **image and speech recognition** applications, such as facial recognition, voice recognition, and emotion detection.
- Random Forest is used in **social media analysis** for tasks such as sentiment analysis, predicting user behavior, and recommending content.
- In **healthcare**, Random Forest can be used for tasks such as predicting disease diagnoses, identifying high-risk patients, and determining treatment plans.

Boosting

Definition and Idea

- Boosting is an ensemble learning method that combines a set of weak learners into a strong learner to minimize training errors.
- In boosting, a random sample of data is chosen, fitted with a model, and then sequentially trained. In other words, each model aims to make up for the shortcomings of the one before it.
- Intuitively, we do not have a super learner, but many bad learners. These bad learners are combined to obtain a strong learner with lower bias.
- So, Boosting is used to shift the models from **high** bias $\longrightarrow$ **low** bias.

- Boosting algorithms can differ in how they create and aggregate weak learners during the sequential process.
- There are three popular types of boosting: AdaBoost, Gradient Boosting and XGBoost.
- The one discussed in the class is **Gradient Boosting**. It works by sequentially adding predictors to an ensemble, each correcting for its predecessor's errors. Each predictor is trained on the residual errors of the previous predictor.
- When the target column is continuous, we use **Gradient Boosting Regressor**; when it is a classification problem, we use **Gradient Boosting Classifier**. The difference between the two is the *Loss function* used.

Gradient Boosting Regressor

Algorithm

- Let us consider a decision tree with regression. We will choose an ensemble consists of **N** decision trees (DT's). The **N here is a hyperparameter**.
- Here, Tree-1 is trained using the feature matrix X and the labels y. The predictions labelled $\hat{y_1}$ are used to determine the training set residual errors r_1.
- Tree-2 is then trained using the feature matrix X and the residual errors r_1 of Tree-1 as labels. The predicted results $\hat{r_1}$ are then used to determine the residual r_2.
- The process is repeated until all the ensemble's N trees are trained.

Gradient Boosting Regressor

Algorithm: Continued

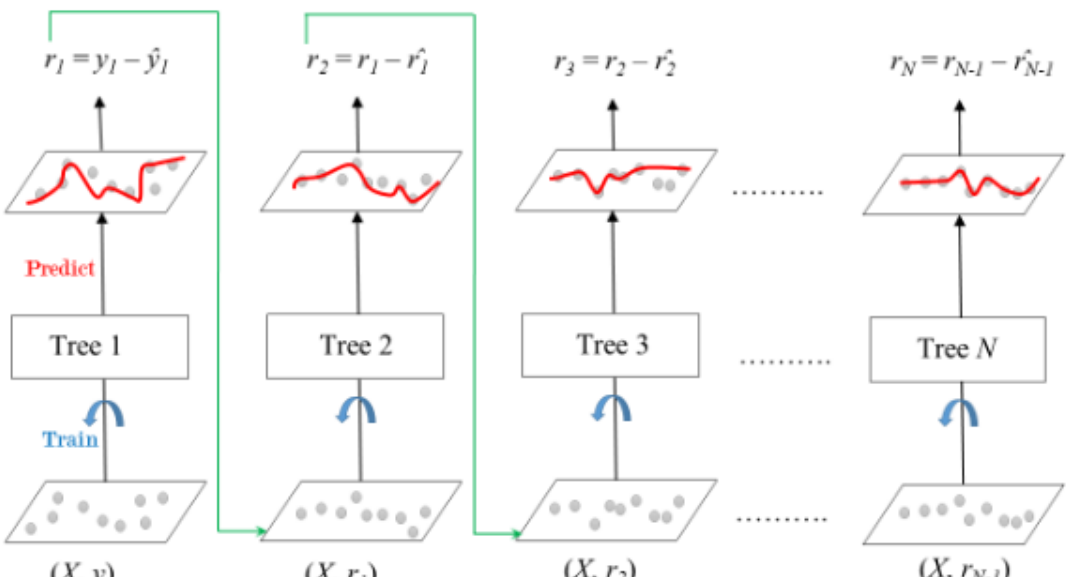


Figure. Gradient Boosted Trees for Regression

Gradient Boosting Regressor

Shrinkage

- A learning rate η, which ranges from $(0, 1)$, is being introduced, which will be multiplied with the prediction of each tree in the ensemble. This process is known as Shrinkage.
- Such shrinking of the prediction is introduced to avoid over-fitting, and thus η can be considered a damping factor.
- There is a trade-off between η and **N** (the number of DT's / estimators). Decreasing learning rate ($\eta \downarrow$) needs to be compensated with increasing estimators (**N** $\uparrow$) to reach specific model performance.

Each tree predicts labels, and this formula gives the final prediction,

$$y_{pred} = \hat{y}_1 + \eta_1 r_1 + \eta_2 r_2 + ... + \eta_N r_N \tag{1}$$

where η_i are the learning rate for the estimators DT_i with residual error r_i .

Example 2.1

Suppose, we were trying to predict the price of a house given their age, square footage and location.

Table. Price of the house

Age	Square Footage	Location	Price
5	1500	5	480
11	2030	12	1090
14	1442	6	350
8	2501	4	1310
12	1300	9	400

- **Step 1**: A decision tree DT_0 with a leaf is created, the predicted value for the variable of interest. Here, it is the price of the house. Then this leaf is made as the baseline to approach the correct solution sequentially.

Gradient Boosting Regressor

Example-Continued

- **Step 2**: The residual r_i is calculated for every sample.
  Residual (r) = Actual Value($\hat{y}$) - Predicted Value(y_{pred})

Table. GradBoost DT_1

Age	Square Footage	Location	Price	Predicted $\hat{y}_1$	Residuals r_1
5	1500	5	480	440	40
11	2030	12	1090	880	210
14	1442	6	350	400	-50
8	2501	4	1310	1010	300
12	1300	9	400	600	-200

- **Step 3:** We then build a decision tree (DT_1) that makes predictions on the residuals r_1 and we obtain $\hat{y}_2$.
- **Step 4:** The process is continued by developing N decision trees and finally attaining the predicted value for each sample with suitable weightage as a learning parameter, as mentioned in 1.
- So, the final prediction of the price of the house y_{pred} is
  y_{pred} = Price predicted from DT_0 + η_1*Residual(r_1) from DT_1 + ... + η_N*Residual(r_N) from DT_N
  where η are the learning rate.

Gradient Boosting Regressor

Example-Continued

Table. Boosting DT_2

Age	Square Footage	Location	Price	Predicted $\hat{y}_1$	Residuals r_1	Predicted $\hat{y}_2$	Residuals r_2
5	1500	5	480	440	40	20	20
11	2030	12	1090	880	210	200	10
14	1442	6	350	400	-50	0	-50
8	2501	4	1310	1010	300	240	60
12	1300	9	400	600	-200	-50	-150

Table. Boosting final prediction after N Decision Trees

Age	Square Footage	Location	Price	Predicted $\hat{y}_1$	Residuals r_1	Residuals r_2	...	Price Prediction y_{pred}
5	1500	5	480	440	40	20		478
11	2030	12	1090	880	210	10		1091
14	1442	6	350	400	-50	-50		335
8	2501	4	1310	1010	300	60		1311
12	1300	9	400	600	-200	-150		400

Boosting: Key Benefits and Challenges

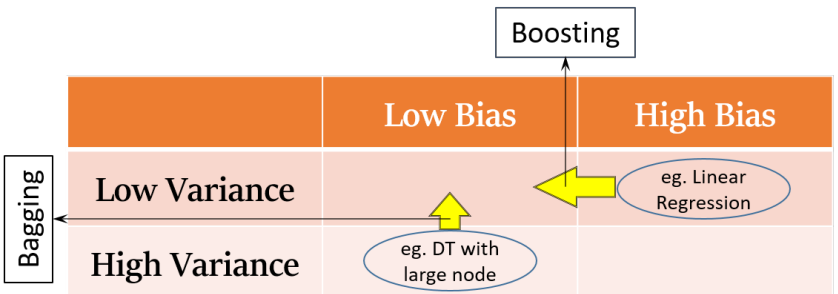
- **Benefits:**
  - **Reduction of bias:** Boosting algorithms sequentially combine several weak learners, iteratively improving on observations. This method can aid in lowering high bias, which is frequently present in logistic regression models and shallow decision trees.
  - **Computational Efficiency:** Boosting algorithms can aid in reducing dimensionality and improving computational efficiency because they only choose features during training that improve their predictive power.
- **Challenges:**
  - **Overfitting:** There has been evidences boosting sometimes lead to overfitting.
  - **Intense computation:** Scaling sequential training in boosting is challenging. Boosting models can be computationally expensive because each estimator builds on its predecessors.

Boosting: Applications

- When **predicting medical data**, such as cardiovascular risk factors and cancer patient survival rates, boosting is used to reduce prediction errors.
- The Viola-Jones boosting algorithm is used for **image retrieval**, while search engines use gradient boosted regression trees for **page rankings**.
- Deep learning models and boosting are combined to automate important tasks like **fraud detection**, **pricing analysis**, and more.

Bagging Vs Boosting

Summary



Bagging

- Used on weak learners that exhibit high variance and low bias (HVLB)
- The weak learners are trained in parallel
- Each learner receives equal weight.
- Bagging can be used to avoid overfitting.

Boosting

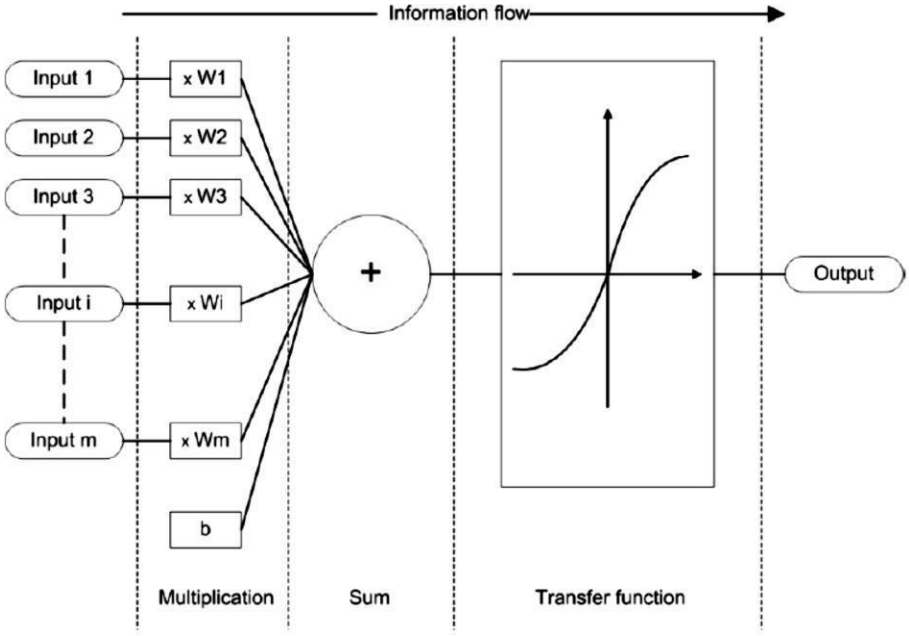
- Used on weak learners that exhibit low variance and high bias (LVHB)
- The weak Learners are learned Sequentially.
- Each learner is weighted differently according to their performance.
- Boosting can be used to lower bias.

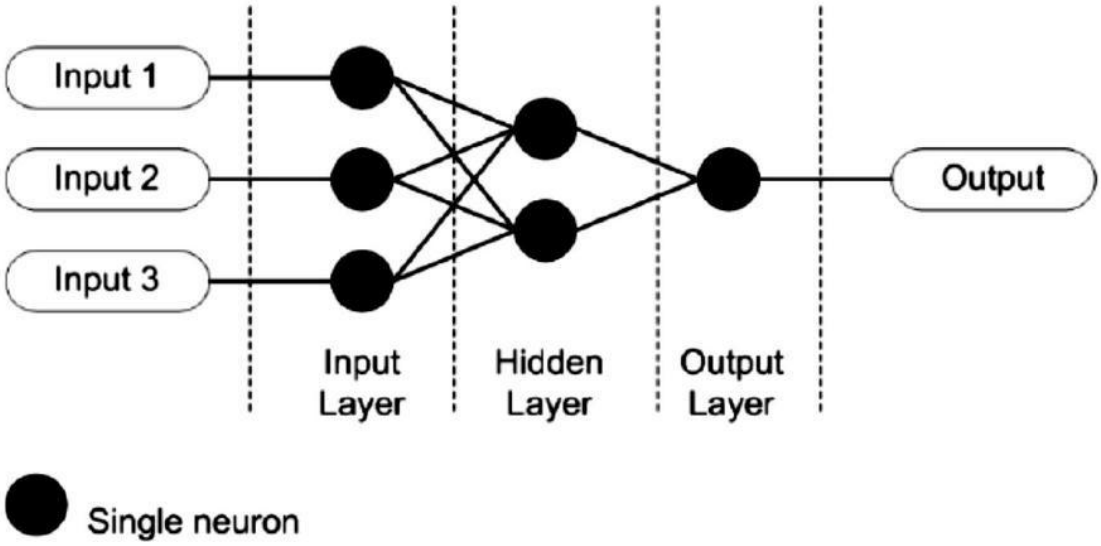
UNIT-1

NEURAL NETWORKS-1

WHAT IS ARTIFICIAL NEURAL NETWORK?

An Artificial Neural Network (ANN) is a mathematical model that tries to simulate the structure and functionalities of biological neural networks. Basic building block of every artificial neural network is artificial neuron, that is, a simple mathematical model (function). Such a model has three simple sets of rules: multiplication, summation and activation. At the entrance of artificial neuron the inputs are weighted what means that every input value is multiplied with individual weight. In the middle section of artificial neuron is sum function that sums all weighted inputs and bias. At the exit of artificial neuron the sum of previously weighted inputs and bias is passing through activation function that is also called transfer function.





BIOLOGICAL NEURON STRUCTURE AND FUNCTIONS.

A neuron, or nerve cell, is an electrically excitable cell that communicates with other cells via specialized connections called synapses. It is the main component of nervous tissue. Neurons are typically classified into three types based on their function. Sensory neurons respond to stimuli such as touch, sound, or light that affect the cells of the sensory organs, and they send signals to the spinal cord or brain. Motor neurons receive signals from the brain and spinal cord to control everything from muscle contractions to glandular output. Interneurons connect neurons to other neurons within the same region of the brain or spinal cord. A group of connected neurons is called a neural circuit.

A typical neuron consists of a cell body (soma), dendrites, and a single axon. The soma is usually compact. The axon and dendrites are filaments that extrude from it. Dendrites typically branch profusely and extend a few hundred micrometers from the soma. The axon leaves the soma at a swelling called the axon hillock, and travels for as far as 1 meter in humans or more in other species. It branches but usually maintains a constant diameter. At the farthest tip of the axon's branches are axon terminals, where the neuron can transmit a signal across the synapse to another cell. Neurons may lack dendrites or have no axon. The term neurite is used to describe either a dendrite or an axon, particularly when the cell is undifferentiated.

The soma is the body of the neuron. As it contains the nucleus, most protein synthesis occurs here. The nucleus can range from 3 to 18 micrometers in diameter.

The dendrites of a neuron are cellular extensions with many branches. This overall shape and structure is referred to metaphorically as a dendritic tree. This is where the majority of input to the neuron occurs via the dendritic spine.

The axon is a finer, cable-like projection that can extend tens, hundreds, or even tens of thousands of times the diameter of the soma in length. The axon primarily carries nerve signals away from the soma, and carries some types of information back to it. Many neurons have only one axon, but this axon may—and usually will—undergo extensive branching, enabling communication with many target cells. The part of the axon where it emerges from the soma is called the axon hillock. Besides being an anatomical structure, the axon hillock also has the greatest density of voltage-dependent sodium channels. This makes it the most easily excited part of the neuron and the spike initiation zone for the axon. In electrophysiological terms, it has the most negative threshold potential.

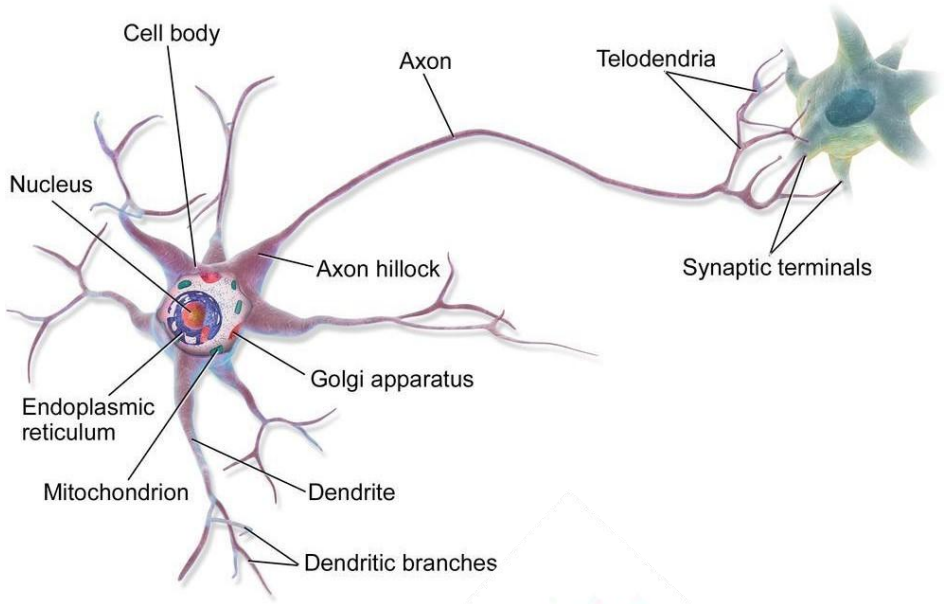
While the axon and axon hillock are generally involved in information outflow, this region can also receive input from other neurons.

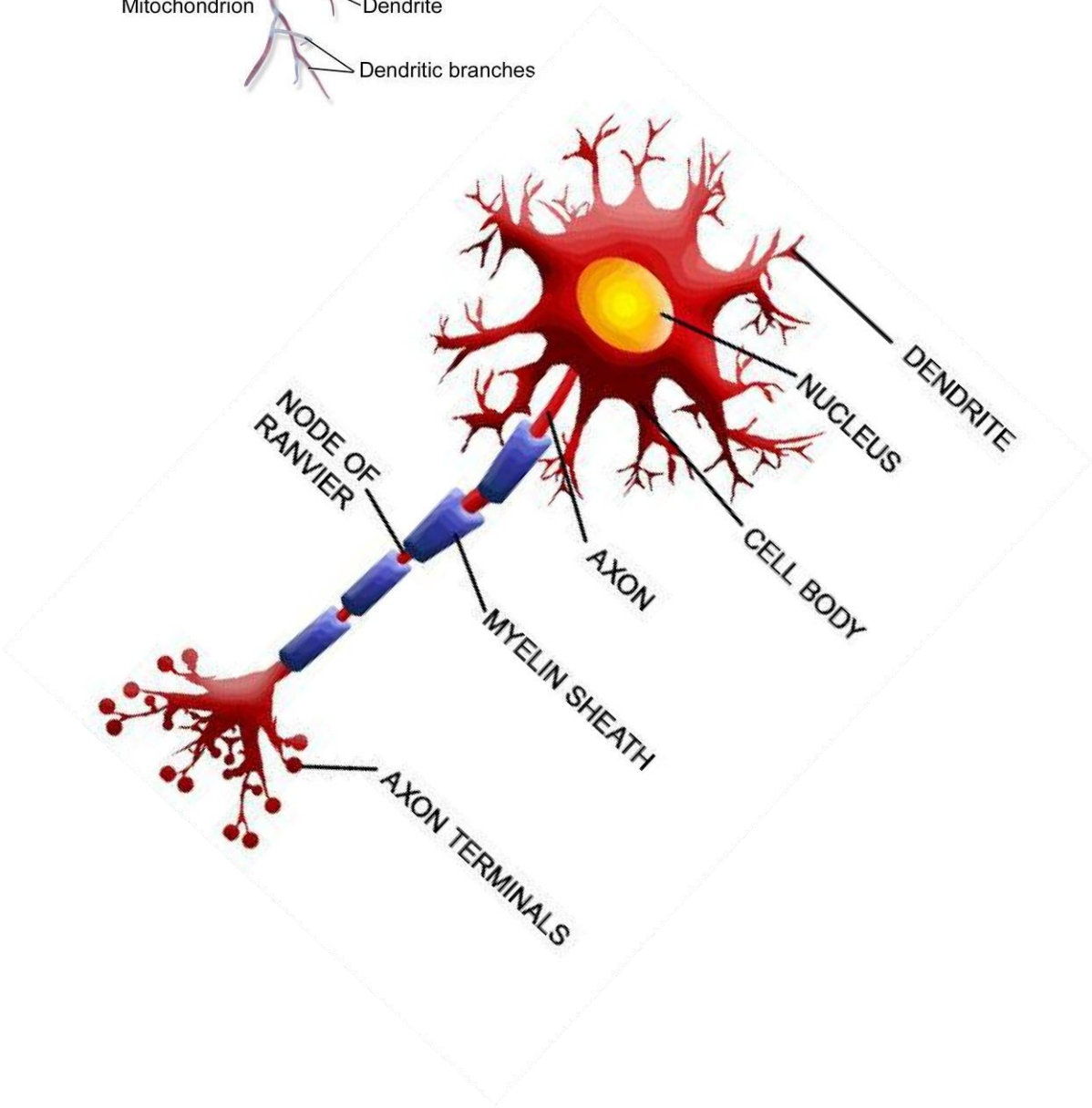
The axon terminal is found at the end of the axon farthest from the soma and contains synapses. Synaptic boutons are specialized structures where neurotransmitter chemicals are released to communicate with target neurons. In addition to synaptic boutons at the axon terminal, a neuron may have en passant boutons, which are located along the length of the axon.

Most neurons receive signals via the dendrites and soma and send out signals down the axon. At the majority of synapses, signals cross from the axon of one neuron to a dendrite of another. However, synapses can connect an axon to another axon or a dendrite to another dendrite. The signaling process is partly electrical and partly chemical. Neurons are electrically excitable, due to maintenance of voltage gradients across their membranes. If the voltage changes by a large amount over a short interval, the neuron generates an all-or-nothing electrochemical pulse called an action potential. This potential travels

rapidly along the axon, and activates synaptic connections as it reaches them. Synaptic signals may be excitatory or inhibitory, increasing or reducing the net voltage that reaches the soma.

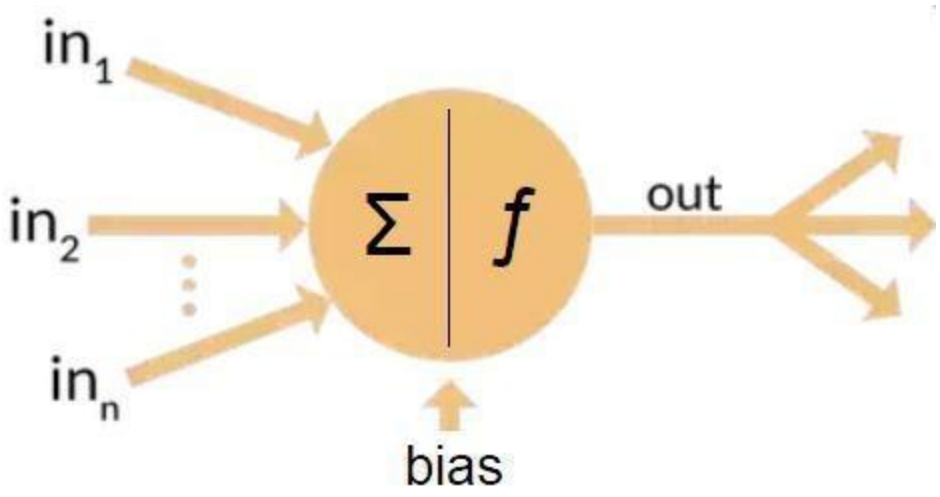
In most cases, neurons are generated by neural stem cells during brain development and childhood. Neurogenesis largely ceases during adulthood in most areas of the brain. However, strong evidence supports generation of substantial numbers of new neurons in the hippocampus and olfactory bulb.

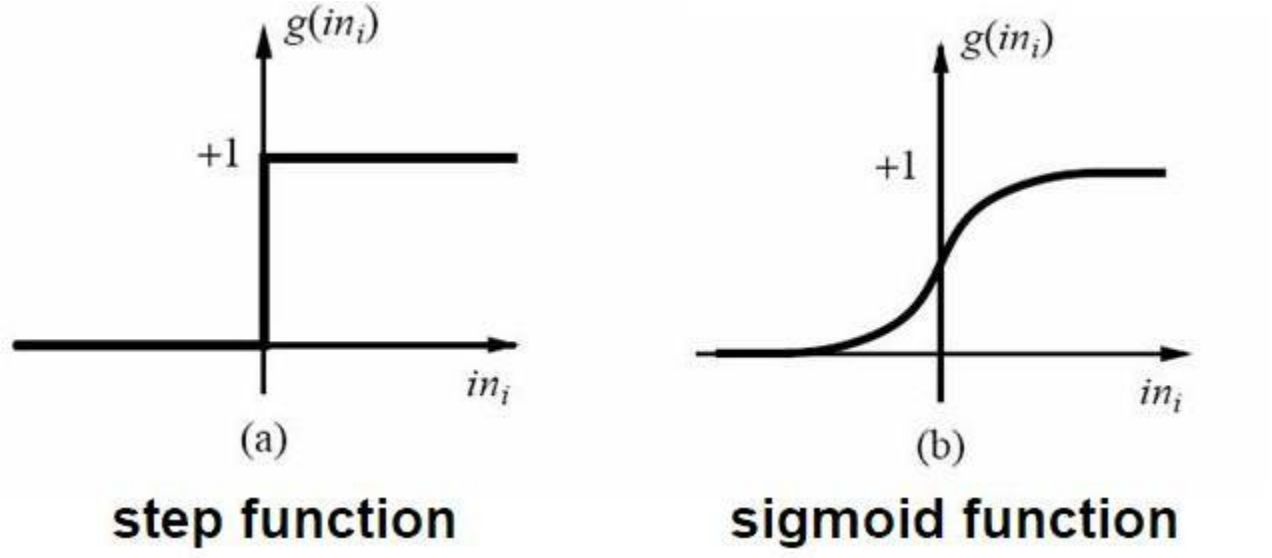




STRUCTURE AND FUNCTIONS OF ARTIFICIAL NEURON.

An artificial neuron is a mathematical function conceived as a model of biological neurons, a neural network. Artificial neurons are elementary units in an artificial neural network. The artificial neuron receives one or more inputs (representing excitatory postsynaptic potentials and inhibitory postsynaptic potentials at neural dendrites) and sums them to produce an output (or activation, representing a neuron's action potential which is transmitted along its axon). Usually each input is separately weighted, and the sum is passed through a non-linear function known as an activation function or transfer function. The transfer functions usually have a sigmoid shape, but they may also take the form of other non-linear functions, piecewise linear functions, or step functions. They are also often monotonically increasing, continuous, differentiable and bounded. The thresholding function has inspired building logic gates referred to as threshold logic; applicable to building logic circuits resembling brain processing. For example, new devices such as memristors have been extensively used to develop such logic in recent times.





STATE THE MAJOR DIFFERENCES BETWEEN BIOLOGICAL AND ARTIFICIANEURAL NETWORKS

1. Size: Our brain contains about 86 billion neurons and more than a 100 synapses (connections). The number of "neurons" in artificial networks is much less than that.

2. Signal transport and processing: The human brain works asynchronously, ANNs work synchronously.

3. Processing speed: Single biological neurons are slow, while standard neurons in ANNs are fast.

4. Topology: Biological neural networks have complicated topologies, while ANNs are often in a tree structure.

5. Speed: certain biological neurons can fire around 200 times a second on average. Signals travel at different speeds depending on the type of the nerve impulse, ranging from 0.61 m/s up to 119 m/s. Signal travel speeds also vary from person to person depending on their sex, age, height, temperature, medical condition, lack of sleep etc. Information in artificial neurons is carried over by the continuous, floating point number values of synaptic weights. There are no refractory periods for artificial neural networks (periods while it is impossible to send another action potential, due to the sodium channels being lock shut) and artificial neurons do not experience "fatigue": they are functions that can be calculated as many times and as fast as the computer architecture would allow.

6. Fault-tolerance: biological neuron networks due to their topology are also fault-tolerant. Artificial neural networks are not modeled for fault tolerance or self regeneration (similarly to fatigue, these ideas are not applicable to matrix operations), though recovery is possible by saving the current state (weight values) of the model and continuing the training from that save state.

7. Power consumption: the brain consumes about 20% of all the human body's energy — despite it's large cut, an adult brain operates on about 20 watts (barely enough to dimly light a bulb) being extremely efficient. Taking into account how humans can still operate for a while, when only given some c-vitamin rich lemon juice and beef tallow, this is quite remarkable. For benchmark: a single Nvidia GeForce Titan X GPU runs on 250 watts alone, and requires a power supply. Our machines are way less efficient than biological systems. Computers also generate a lot of heat when used, with consumer GPUs operating safely between 50–80°Celsius instead of 36.5–37.5 °C.

8. Learning: we still do not understand how brains learn, or how redundant connections store and recall information. By learning, we are building on information that is already stored in the brain. Our knowledge deepens by repetition and during sleep, and tasks that once required a focus can be executed automatically once mastered. Artificial neural networks in the other hand, have a predefined model, where no further neurons or connections can be added or removed. Only the weights of the connections (and biases representing thresholds) can change during training. The networks start with random weight values and will slowly try to reach a point where further changes in the weights would no longer improve performance. Biological networks usually don't stop / start learning. ANNs have different fitting (train) and prediction (evaluate) phases.

9. Field of application: ANNs are specialized. They can perform one task. They might be perfect at playing chess, but they fail at playing go (or vice versa). Biological neural networks can learn completely new tasks.

10. Training algorithm: ANNs use Gradient Descent for learning. Human brains use something different (but we don't know what).

BRIEFLY EXPLAIN THE BASIC BUILDING BLOCKS OF ARTIFICIAL NEURAL NETWORKS.

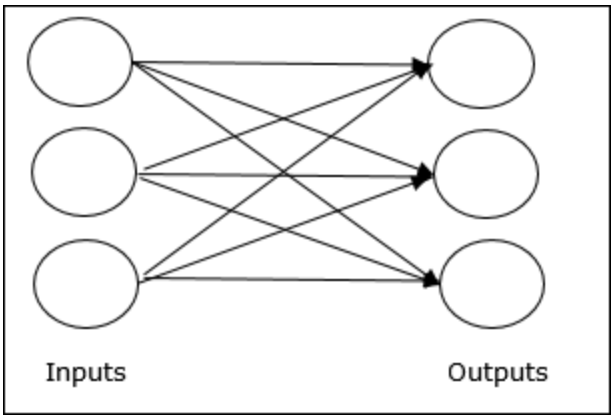
Processing of ANN depends upon the following three building blocks:

1. Network Topology
2. Adjustments of Weights or Learning
3. Activation Functions

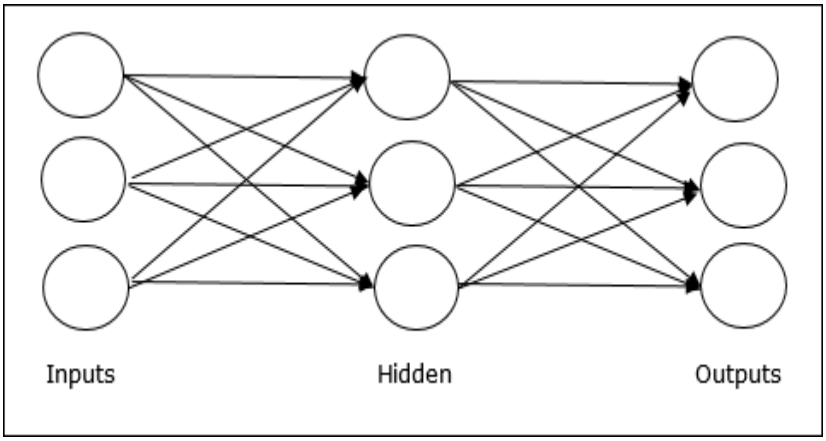
1. Network Topology: A network topology is the arrangement of a network along with its nodes and connecting lines. According to the topology, ANN can be classified as the following kinds:

A. Feed forward Network: It is a non-recurrent network having processing units/nodes in layers and all the nodes in a layer are connected with the nodes of the previous layers. The connection has different weights upon them. There is no feedback loop means the signal can only flow in one direction, from input to output. It may be divided into the following two types:

- **Single layer feed forward network:** The concept is of feed forward ANN having only one weighted layer. In other words, we can say the input layer is fully connected to the output layer.

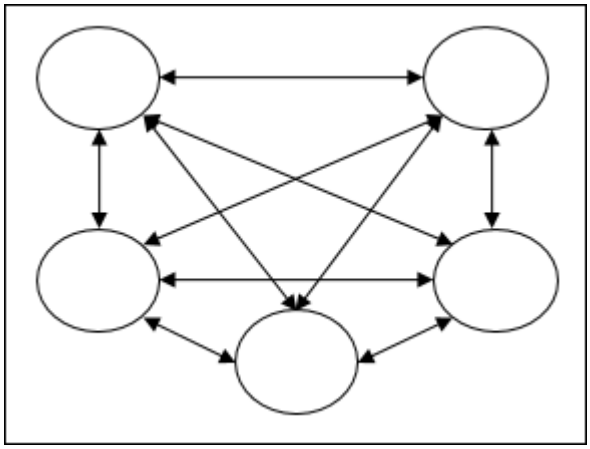


- **Multilayer feed forward network:** The concept is of feed forward ANN having more than one weighted layer. As this network has one or more layers between the input and the output layer, it is called hidden layers.

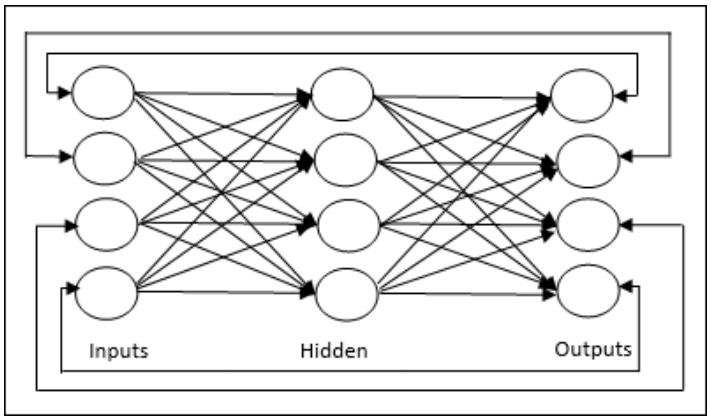


B. Feedback Network: As the name suggests, a feedback network has feedback paths, which means the signal can flow in both directions using loops. This makes it a non-linear dynamic system, which changes continuously until it reaches a state of equilibrium. It may be divided into the following types:

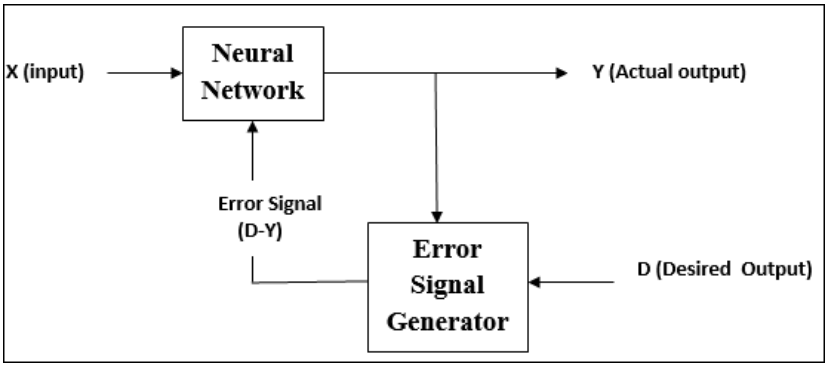
- **Recurrent networks:** They are feedback networks with closed loops. Following are the two types of recurrent networks.
- **Fully recurrent network:** It is the simplest neural network architecture because all nodes are connected to all other nodes and each node works as both input and output.



- **Jordan network** – It is a closed loop network in which the output will go to the input again as feedback as shown in the following diagram.

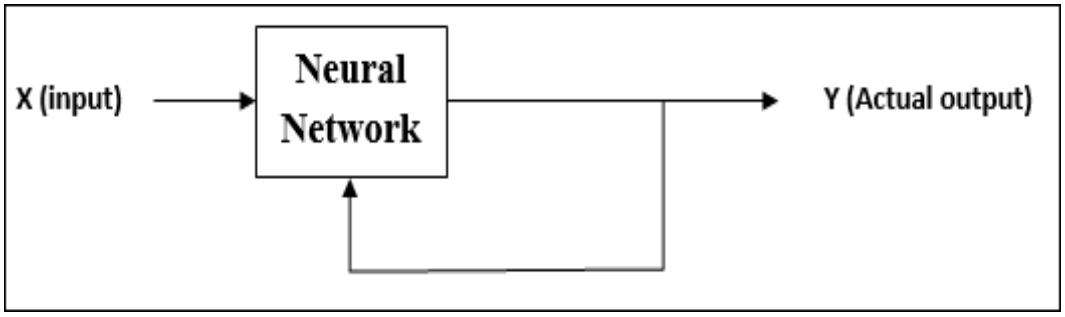


2. Adjustments of Weights or Learning: Learning, in artificial neural network, is the method of modifying the weights of connections between the neurons of a specified network. Learning in ANN can be classified into three categories namely supervised learning, unsupervised learning, and reinforcement learning.

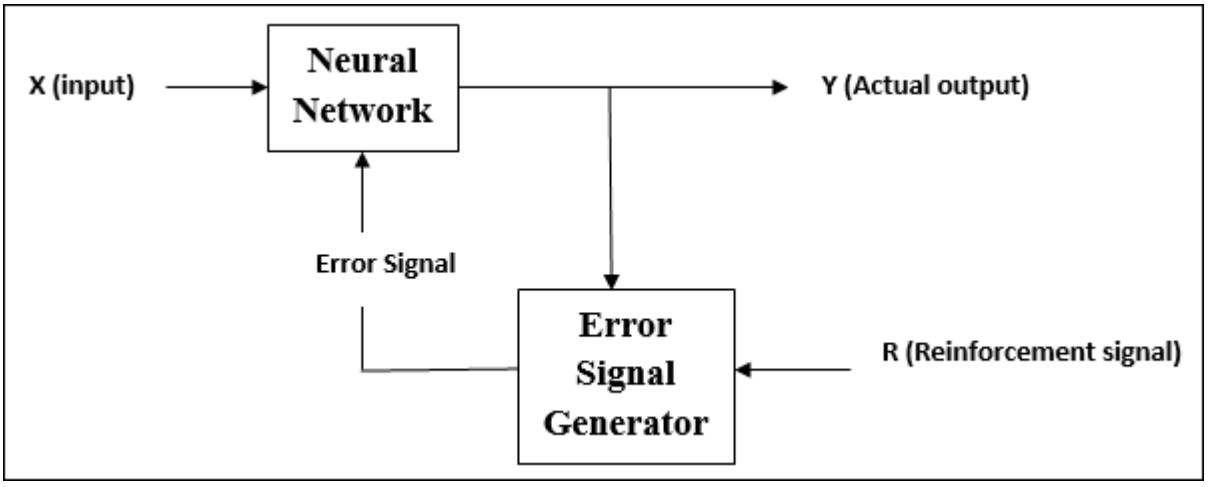


Supervised Learning: As the name suggests, this type of learning is done under the supervision of a teacher. This learning process is dependent. During the training of ANN under supervised learning, the input vector is presented to the network, which will give an output vector. This output vector is compared with the desired output vector. An error signal is generated, if there is a difference between the actual output and the desired output vector. On the basis of this error signal, the weights are adjusted until the actual output is matched with the desired output.

Unsupervised Learning: As the name suggests, this type of learning is done without the supervision of a teacher. This learning process is independent. During the training of ANN under unsupervised learning, the input vectors of similar type are combined to form clusters. When a new input pattern is applied, then the neural network gives an output response indicating the class to which the input pattern belongs. There is no feedback from the environment as to what should be the desired output and if it is correct or incorrect. Hence, in this type of learning, the network itself must discover the patterns and features from the input data, and the relation for the input data over the output.



Reinforcement Learning: As the name suggests, this type of learning is used to reinforce or strengthen the network over some critic information. This learning process is similar to supervised learning, however we might have very less information. During the training of network under reinforcement learning, the network receives some feedback from the environment. This makes it somewhat similar to supervised learning. However, the feedback obtained here is evaluative not instructive, which means there is no teacher as in supervised learning. After receiving the feedback, the network performs adjustments of the weights to get better critic information in future.



3. **Activation Functions:** An activation function is a mathematical equation that determines the output of each element (perceptron or neuron) in the neural network. It takes in the input from each neuron and transforms it into an output, usually between one and zero or between -1 and one. It may be defined as the extra force or effort applied over the input to obtain an exact output. In ANN, we can also apply activation functions over the input to get the exact output. Followings are some activation functions of interest:

i) Linear Activation Function: It is also called the identity function as it performs no input editing. It can be defined as: $F(x) = x$

ii) Sigmoid Activation Function: It is of two type as follows –

- **Binary sigmoidal function:** This activation function performs input editing between 0 and 1. It is positive in nature. It is always bounded, which means its output cannot be less than 0 and more than 1. It is also strictly increasing in nature, which means more the input higher would be the output. It can be defined as

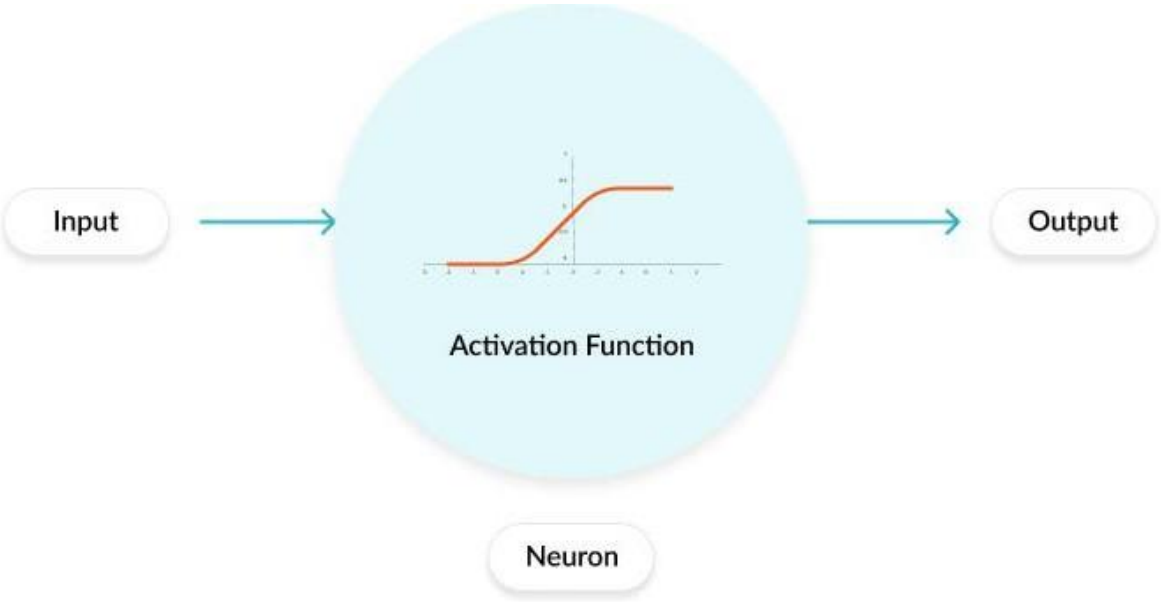
$$F(x)=sigm(x)=11+exp(−x)F(x)=sigm(x)=11+exp(−x)$$

- **Bipolar sigmoidal function:** This activation function performs input editing between -1 and 1. It can be positive or negative in nature. It is always bounded, which means its output cannot be less than -1 and more than 1. It is also strictly increasing in nature like sigmoid function. It can be defined as

$$F(x)=sigm(x)=21+exp(−x)−1=1−exp(x)1+exp(x)$$

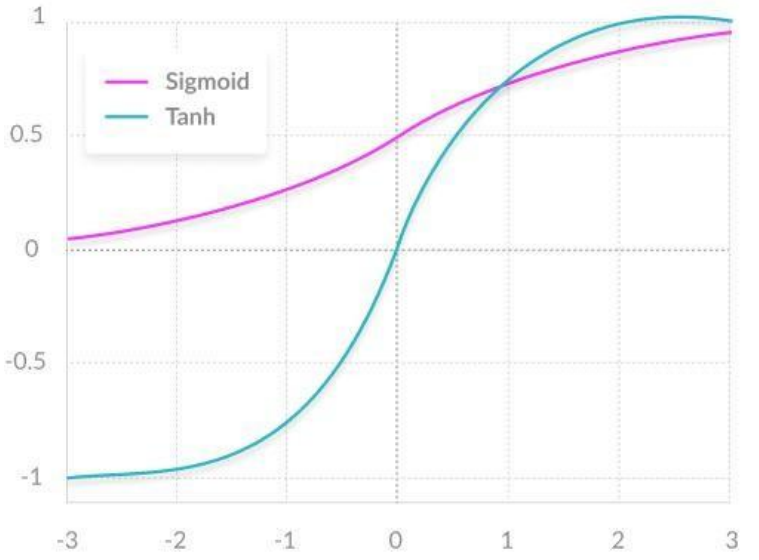
WHAT IS A NEURAL NETWORK ACTIVATION FUNCTION?

In a neural network, inputs, which are typically real values, are fed into the neurons in the network. Each neuron has a weight, and the inputs are multiplied by the weight and fed into the activation function. Each neuron's output is the input of the neurons in the next layer of the network, and so the inputs cascade through multiple activation functions until eventually, the output layer generates a prediction. Neural networks rely on nonlinear activation functions—the derivative of the activation function helps the network learn through the backpropagation process.



SOME COMMON ACTIVATION FUNCTIONS INCLUDE THE FOLLOWING:

1. **The sigmoid function** has a smooth gradient and outputs values between zero and one. For very high or low values of the input parameters, the network can be very slow to reach a prediction, called the *vanishing gradient* problem.
2. **The TanH function** is zero-centered making it easier to model inputs that are strongly negative strongly positive or neutral.
3. **The ReLu function** is highly computationally efficient but is not able to process inputs that approach zero or negative.
4. **The Leaky ReLu** function has a small positive slope in its negative area, enabling it to process zero or negative values.
5. **The Parametric ReLu** function allows the negative slope to be learned, performing backpropagation to learn the most effective slope for zero and negative input values.
6. **Softmax** is a special activation function use for output neurons. It normalizes outputs for each class between 0 and 1, and returns the probability that the input belongs to a specific class.
7. **Swish** is a new activation function discovered by Google researchers. It performs better than ReLu with a similar level of computational efficiency.



Two common neural network activation functions - Sigmoid and Tanh

APPLICATIONS OF ANN

1. Data Mining: Discovery of meaningful patterns (knowledge) from large volumes of data.
2. Expert Systems: A computer program for decision making that simulates thought process of a human expert.
3. Fuzzy Logic: Theory of approximate reasoning.
4. Artificial Life: Evolutionary Computation, Swarm Intelligence.
5. Artificial Immune System: A computer program based on the biological immune system.
6. Medical: At the moment, the research is mostly on modelling parts of the human body and recognizing diseases from various scans (e.g. cardiograms, CAT scans, ultrasonic scans, etc.).Neural networks are ideal in recognizing diseases using scans since there is no need to provide a specific algorithm on how to identify the disease. Neural networks learn by example so the details of how to recognize the disease are not needed. What is needed is a set of examples that are representative of all the variations of the disease. The quantity of examples is not as important as the 'quantity'. The examples need to be selected very carefully if the system is to perform reliably and efficiently.

7. Computer Science: Researchers in quest of artificial intelligence have created spin offs like dynamic programming, object oriented programming, symbolic programming, intelligent storage management systems and many more such tools. The primary goal of creating an artificial intelligence still remains a distant dream but people are getting an idea of the ultimate path, which could lead to it.
8. Aviation: Airlines use expert systems in planes to monitor atmospheric conditions and system status. The plane can be put on autopilot once a course is set for the destination.
9. Weather Forecast: Neural networks are used for predicting weather conditions. Previous data is fed to a neural network, which learns the pattern and uses that knowledge to predict weather patterns.
10. Neural Networks in business: Business is a diverted field with several general areas of specialization such as accounting or financial analysis. Almost any neural network application would fit into one business area or financial analysis.
11. There is some potential for using neural networks for business purposes, including resource allocation and scheduling.
12. There is also a strong potential for using neural networks for database mining, which is, searching for patterns implicit within the explicitly stored information in databases. Most of the funded work in this area is classified as proprietary. Thus, it is not possible to report on the full extent of the work going on. Most work is applying neural networks, such as the Hopfield-Tank network for optimization and scheduling.
13. Marketing: There is a marketing application which has been integrated with a neural network system. The Airline Marketing Tactician (a trademark abbreviated as AMT) is a computer system made of various intelligent technologies including expert systems. A feed forward neural network is integrated with the AMT and was trained using back-propagation to assist the marketing control of airline seat allocations. The adaptive neural approach was amenable to rule expression. Additionally, the application's environment changed rapidly and constantly, which required a continuously adaptive solution.
14. Credit Evaluation: The HNC company, founded by Robert Hecht-Nielsen, has developed several neural network applications. One of them is the Credit Scoring system which increases the profitability of the existing model up to 27%. The HNC neural systems were also applied to mortgage screening. A neural network automated mortgage insurance under writing system was developed by the Nestor Company. This system was trained with 5048 applications of which 2597 were certified. The data related to property and borrower qualifications. In a conservative mode the system agreed on the under writers on 97% of the cases. In the liberal model the system agreed 84% of the cases. This is system run on an Apollo DN3000 and used 250K memory while processing a case file in approximately 1 sec.

ADVANTAGES OF ANN

1. Adaptive learning: An ability to learn how to do tasks based on the data given for training or initial experience.
2. Self-Organisation: An ANN can create its own organisation or representation of the information it receives during learning time.
3. Real Time Operation: ANN computations may be carried out in parallel, and special hardware devices are being designed and manufactured which take advantage of this capability.
4. Pattern recognition: is a powerful technique for harnessing the information in the data and generalizing about it. Neural nets learn to recognize the patterns which exist in the data set.
5. The system is developed through learning rather than programming.. Neural nets teach themselves the patterns in the data freeing the analyst for more interesting work.

6. Neural networks are flexible in a changing environment. Although neural networks may take some time to learn a sudden drastic change they are excellent at adapting to constantly changing information.
7. Neural networks can build informative models whenever conventional approaches fail. Because neural networks can handle very complex interactions they can easily model data which is too difficult to model with traditional approaches such as inferential statistics or programming logic.
8. Performance of neural networks is at least as good as classical statistical modelling, and better on most problems. The neural networks build models that are more reflective of the structure of the data in significantly less time.

LIMITATIONS OF ANN

In this technological era everything has Merits and some Demerits in others words there is a Limitation with every system which makes this ANN technology weak in some points. The various Limitations of ANN are:-

1) ANN is not a daily life general purpose problem solver.
2) There is no structured methodology available in ANN.
3) There is no single standardized paradigm for ANN development.
4) The Output Quality of an ANN may be unpredictable.
5) Many ANN Systems does not describe how they solve problems.
6) Black box Nature
7) Greater computational burden.
8) Proneness to over fitting.
9) Empirical nature of model development.

ARTIFICIAL NEURAL NETWORK CONCEPTS/TERMINOLOGY

Here is a glossary of basic terms you should be familiar with before learning the details of neural networks.

Inputs: Source data fed into the neural network, with the goal of making a decision or prediction about the data. Inputs to a neural network are typically a set of real values; each value is fed into one of the neurons in the input layer.

Training Set: A set of inputs for which the correct outputs are known, used to train the neural network.

Outputs : Neural networks generate their predictions in the form of a set of real values or boolean decisions. Each output value is generated by one of the neurons in the output layer.

Neuron/perceptron: The basic unit of the neural network. Accepts an input and generates a prediction.

Each neuron accepts part of the input and passes it through the activation function. Common activation functions are sigmoid, TanH and ReLu. Activation functions help generate output values within an acceptable range, and their non-linear form is crucial for training the network.

Weight Space: Each neuron is given a numeric weight. The weights, together with the activation function, define each neuron's output. Neural networks are trained by fine-tuning weights, to discover the optimal set of weights that generates the most accurate prediction.

Forward Pass: The forward pass takes the inputs, passes them through the network and allows each neuron to react to a fraction of the input. Neurons generate their outputs and pass them on to the next layer, until eventually the network generates an output.

Error Function: Defines how far the actual output of the current model is from the correct output. When training the model, the objective is to minimize the error function and bring output as close as possible to the correct value.

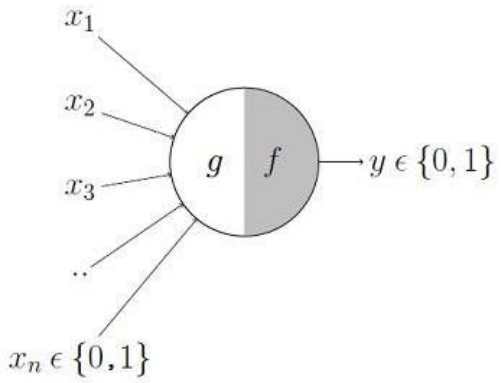
Backpropagation: In order to discover the optimal weights for the neurons, we perform a backward pass, moving back from the network's prediction to the neurons that generated that prediction. This is called backpropagation. Backpropagation tracks the derivatives of the activation functions in each successive neuron, to find weights that bring the loss function to a minimum, which will generate the best prediction. This is a mathematical process called *gradient descent*.

Bias and Variance: When training neural networks, like in other machine learning techniques, we try to balance between bias and variance. Bias measures how well the model fits the training set—able to correctly predict the known outputs of the training examples. Variance measures how well the model works with unknown inputs that were not available during training. Another meaning of bias is a "bias neuron" which is used in every layer of the neural network. The bias neuron holds the number 1, and makes it possible to move the activation function up, down, left and right on the number graph.

Hyperparameters: A hyper parameter is a setting that affects the structure or operation of the neural network. In real deep learning projects, tuning hyper parameters is the primary way to build a network that provides accurate predictions for a certain problem. Common hyper parameters include the number of hidden layers, the activation function, and how many times (epochs) training should be repeated.

MCCULLOGH-PITTS MODEL

In 1943 two electrical engineers, Warren McCullogh and Walter Pitts, published the first paper describing what we would call a neural network.



It may be divided into 2 parts. The first part, g takes an input, performs an aggregation and based on the aggregated value the second part, f makes a decision. Let us suppose that I want to predict my own decision, whether to watch a random football game or not on TV. The inputs are all boolean i.e., {0,1} and my output variable is also boolean {0: Will watch it, 1: Won't watch it}.

So, x_1 could be 'is Indian Premier League On' (I like Premier League more)

x_2 could be 'is it a knockout game (I tend to care less about the league level matches)

x_3 could be 'is Not Home' (Can't watch it when I'm in College. Can I?)

x_4 could be 'is my favorite team playing' and so on.

These inputs can either be excitatory or inhibitory. Inhibitory inputs are those that have maximum effect on the decision making irrespective of other inputs i.e., if x_3 is 1 (not home) then my output will always be 0 i.e., the neuron will never fire, so x_3 is an inhibitory input. Excitatory inputs are NOT the ones that will make the neuron fire on their own but they might fire it when combined together. Formally, this is what is going on:

$$g(x_1, x_2, x_3, ..., x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

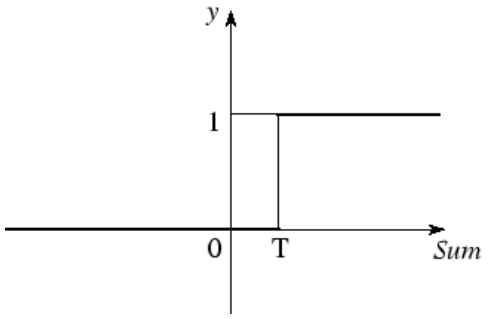
$$\begin{aligned} y = f(g(\mathbf{x})) &= 1 \quad if \quad g(\mathbf{x}) \geq \theta \\ &= 0 \quad if \quad g(\mathbf{x}) < \theta \end{aligned}$$

We can see that g(x) is just doing a sum of the inputs — a simple aggregation. And theta here is called thresholding parameter. For example, if I always watch the game when the sum turns out to be 2 or more, the theta is 2 here. This is called the Thresholding Logic.

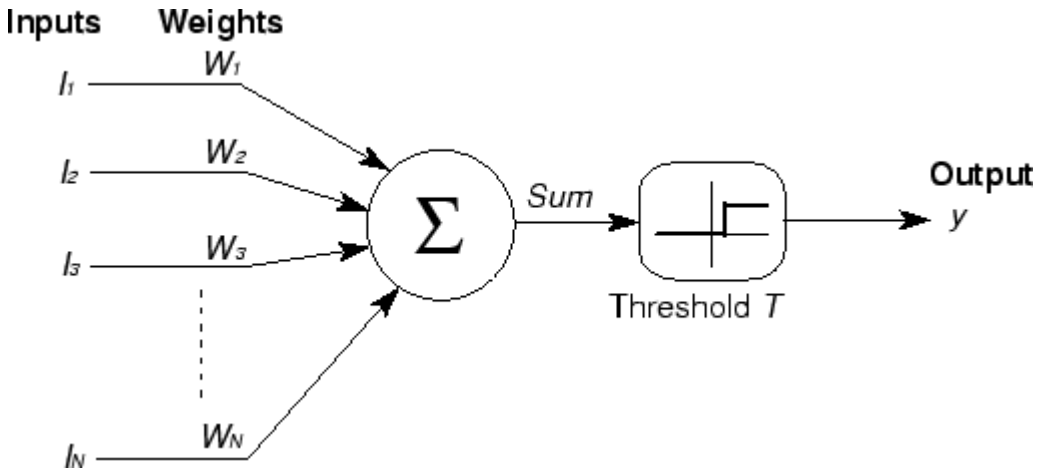
The McCulloch-Pitts neural model is also known as linear threshold gate. It is a neuron of a set of inputs I_1, I_2,I_3,...I_m and one output 'y' . The linear threshold gate simply classifies the set of inputs into two different classes. Thus the output y is binary. Such a function can be described mathematically using these equations:

$$Sum = \sum_{i=1}^{N} I_i W_i, \qquad y = f(Sum).$$

Where, $W_1, W_2, W_3, ..., W_m$ are weight values normalized in the range of either $(0,1)$ or $(-1,1)$ and associated with each input line, Sum is the weighted sum, and T is a threshold constant. The function f is a linear step function at threshold T as shown in figure 2.3. The symbolic representation of the linear threshold gate is shown in figure below.



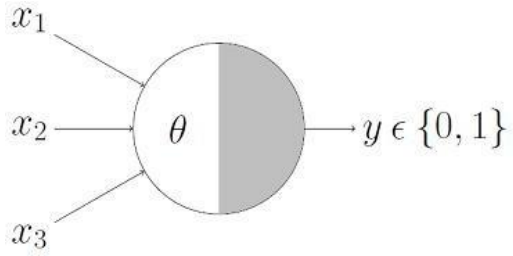
Linear Threshold Function



Symbolic Illustration of Linear Threshold Gate

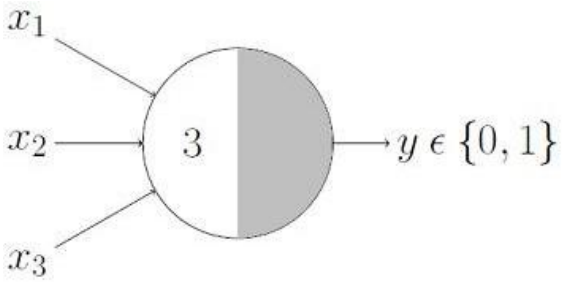
BOOLEAN FUNCTIONS USING McCULLOGH-PITTS NEURON

In any Boolean function, all inputs are Boolean and the output is also Boolean. So essentially, the neuron is just trying to learn a Boolean function.



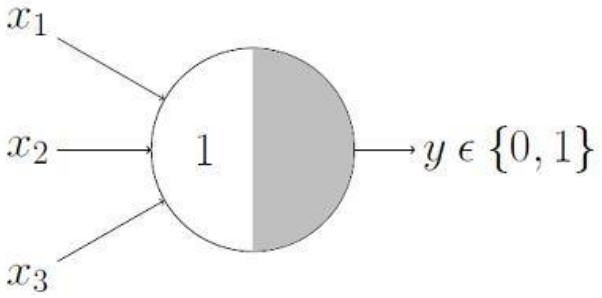
This representation just denotes that, for the boolean inputs x_1, x_2 and x_3 if the g(x) i.e., sum ≥ theta, the neuron will fire otherwise, it won't.

AND Function



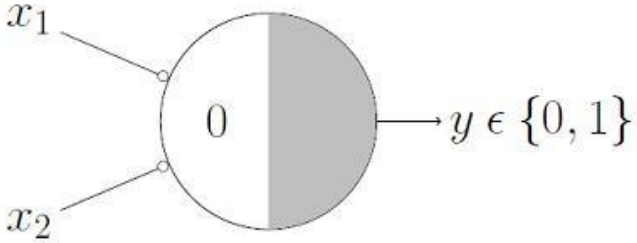
An AND function neuron would only fire when ALL the inputs are ON i.e., g(x) ≥ 3 here.

OR Function



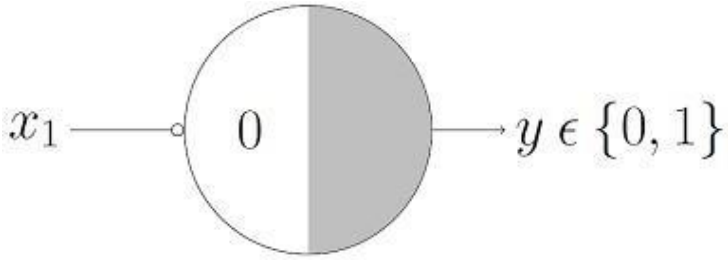
For an OR function neuron would fire if ANY of the inputs is ON i.e., g(x) ≥ 1 here.

NOR Function



For a NOR neuron to fire, we want ALL the inputs to be 0 so the thresholding parameter should also be 0 and we take them all as inhibitory input.

NOT Function



For a NOT neuron, 1 outputs 0 and 0 outputs 1. So we take the input as an inhibitory input and set the thresholding parameter to 0.

We can summarize these rules with the McCullough-Pitts output rule as:

The McCulloch-Pitts model of a neuron is simple yet has substantial computing potential. It also has a precise mathematical definition. However, this model is so simplistic that it only generates a binary output and also the weight and threshold values are fixed. The neural computing algorithm has diverse features for various applications. Thus, we need to obtain the neural model with more flexible computational features.

WHAT ARE THE LEARNING RULES IN ANN?

Learning rule is a method or a mathematical logic. It helps a Neural Network to learn from the existing conditions and improve its performance. Thus learning rules updates the weights and bias levels of a network when a network simulates in a specific data environment. Applying learning rule is an iterative process. It helps a neural network to learn from the existing conditions and improve its performance.

The different learning rules in the Neural network are:

1. Hebbian learning rule – It identifies, how to modify the weights of nodes of a network.
2. Perceptron learning rule – Network starts its learning by assigning a random value to each weight.
3. Delta learning rule – Modification in sympatric weight of a node is equal to the multiplication of error and the input.
4. Correlation learning rule – The correlation rule is the supervised learning.
5. Outstar learning rule – We can use it when it assumes that nodes or neurons in a network arranged in a layer.

1. **Hebbian Learning Rule:** The Hebbian rule was the first learning rule. In 1949 Donald Hebb developed it as learning algorithm of the unsupervised neural network. We can use it to identify how to improve the weights of nodes of a network. The Hebb learning rule assumes that – If two neighbor neurons activated and deactivated at the same time, then the weight connecting these neurons should increase. At the start, values of all weights are set to zero. This learning rule can be used for both soft- and hard-activation functions. Since desired responses of neurons are not used in the learning procedure, this is the unsupervised learning rule. The absolute values of the weights are usually proportional to the learning time, which is undesired.

$$W_{ij} = x_i * x_j$$

Mathematical Formula of Hebb Learning Rule.

2. **Perceptron Learning Rule:** Each connection in a neural network has an associated weight, which changes in the course of learning. According to it, an example of supervised learning, the network starts its learning by assigning a random value to each weight. Calculate the output value on the basis of a set of records for which we can know the expected output value. This is the learning sample that indicates the entire definition. As a result, it is called a learning sample. The network then compares the calculated output value with the expected value. Next calculates an error function ∈, which can be the sum of squares of the errors occurring for each individual in the learning sample which can be computed as:

$$\sum_i \sum_j \left(E_{ij} - O_{ij} \right)^2$$

Mathematical Formula of Perceptron Learning Rule

Perform the first summation on the individuals of the learning set, and perform the second summation on the output units. E_{ij} and O_{ij} are the expected and obtained values of the j^{th} unit for the i^{th} individual. The network then adjusts the weights of the different units, checking each time to see if the error function has increased or decreased. As in a conventional regression, this is a matter of solving a problem of least squares. Since assigning the weights of nodes according to users, it is an example of supervised learning.

3. **Delta Learning Rule:** Developed by Widrow and Hoff, the delta rule, is one of the most common learning rules. It depends on supervised learning. This rule states that the modification in sympatric weight of a node is equal to the multiplication of error and the input. In Mathematical form the delta rule is as follows:

$$\Delta w = \eta (t - y) x_i$$

Mathematical Formula of Delta Learning Rule

For a given input vector, compare the output vector is the correct answer. If the difference is zero, no learning takes place; otherwise, adjusts its weights to reduce this difference. The change in weight from ui to uj is: dwij = r* ai * ej. where r is the learning rate, ai represents the activation of ui and ej is the difference between the expected output and the actual output of uj. If the set of input patterns form an independent set then learn arbitrary associations using the delta rule.

It has seen that for networks with linear activation functions and with no hidden units. The error squared vs. the weight graph is a paraboloid in n-space. Since the proportionality constant is negative, the graph of such a function is concave upward and has the least value. The vertex of this paraboloid represents the point where it reduces the error. The weight vector corresponding to this point is then the ideal weight vector. We can use the delta learning rule with both single output unit and several output units. While applying the delta rule assume that the error can be directly measured. The aim of applying the delta rule is to reduce the difference between the actual and expected output that is the error.

4. **Correlation Learning Rule:** The correlation learning rule based on a similar principle as the Hebbian learning rule. It assumes that weights between responding neurons should be more positive, and weights between neurons with opposite reaction should be more negative. Contrary to the Hebbian rule, the correlation rule is the supervised learning, instead of an actual. The response, oj, the desired response, dj, uses for the weight-change calculation. In Mathematical form the correlation learning rule is as follows:

$$\Delta w_{ij} = \eta x_i d_j$$

Mathematical Formula of Correlation Learning Rule

Where d_j is the desired value of output signal. This training algorithm usually starts with the initialization of weights to zero. Since assigning the desired weight by users, the correlation learning rule is an example of supervised learning.

5. Out Star Learning Rule: We use the Out Star Learning Rule when we assume that nodes or neurons in a network arranged in a layer. Here the weights connected to a certain node should be equal to the desired outputs for the neurons connected through those weights. The out start rule produces the desired response t for the layer of n nodes. Apply this type of learning for all nodes in a particular layer. Update the weights for nodes are as in Kohonen neural networks. In Mathematical form, express the out star learning as follows:

$$w_{jk} = \begin{cases} \eta(y_k - w_{jk}) & \text{if node j wins the competition} \\ 0 & \text{if node j losses the competition} \end{cases}$$

Mathematical Formula of Out Star Learning Rule

This is a supervised training procedure because desired outputs must be known.

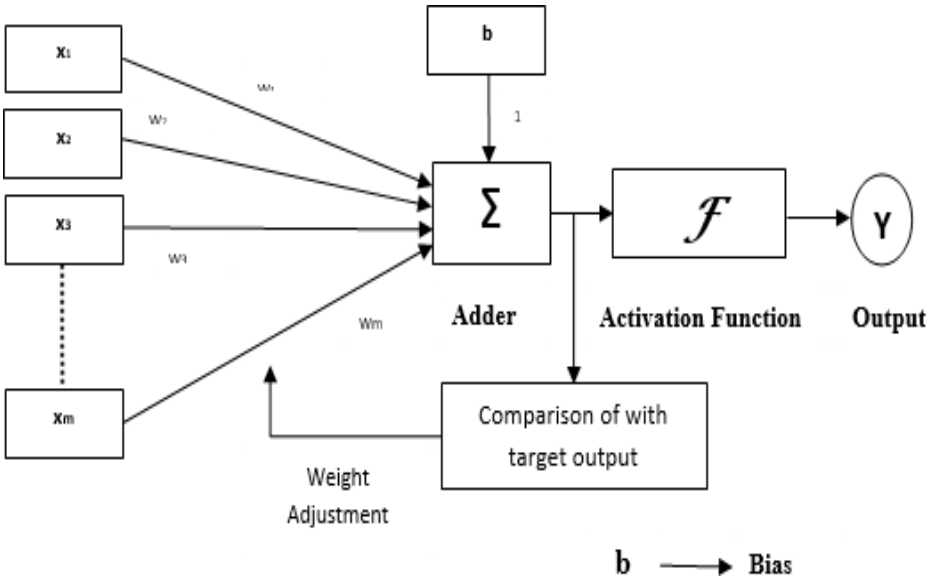
BRIEFLY EXPLAIN THE ADALINE MODEL OF ANN.

ADALINE (Adaptive Linear Neuron or later Adaptive Linear Element) is an early single-layer artificial neural network and the name of the physical device that implemented this network. The network uses memistors. It was developed by Professor Bernard Widrow and his graduate student Ted Hoff at Stanford University in 1960. It is based on the McCulloch–Pitts neuron. It consists of a weight, a bias and a summation function. The difference between Adaline and the standard (McCulloch–Pitts) perceptron is that in the learning phase, the weights are adjusted according to the weighted sum of the inputs (the net). In the standard perceptron, the net is passed to the activation (transfer) function and the function's output is used for adjusting the weights. Some important points about Adaline are as follows:

- It uses bipolar activation function.
- It uses delta rule for training to minimize the Mean-Squared Error (MSE) between the actual output and the desired/target output.
- The weights and the bias are adjustable.

Architecture of ADALINE network: The basic structure of Adaline is similar to perceptron having an extra feedback loop with the help of which the actual output is compared with the desired/target output. After comparison on the basis of training algorithm, the weights and bias will be updated.

Architecture of ADALINE: The basic structure of Adaline is similar to perceptron having an extra feedback loop with the help of which the actual output is compared with the desired/target output. After comparison on the basis of training algorithm, the weights and bias will be updated.



Training Algorithm of ADALINE:

Step 1 – Initialize the following to start the training:

- Weights
- Bias
- Learning rate α

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 – Continue step 3-8 when the stopping condition is not true.

Step 3 – Continue step 4-6 for every bipolar training pair s : t.

Step 4 – Activate each input unit as follows:

$$x_i = s_i \ (i=1 \text{ to } n)$$

Step 5 – Obtain the net input with the following relation:

$$y_{in} = b + \sum_{i}^{n} x_i \ w_i$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output:

$$f(y_{in}) = \begin{cases} 1, \text{if } y_{in} \geq 0 \\ -1, \text{if } y_{in} < 0 \end{cases}$$

Step 7 – Adjust the weight and bias as follows :

Case 1 – if $y \neq t$ then, $w_i(new) = w_i(old) + \alpha(t - y_{in})x_i$

$$b(new) = b(old) + \alpha(t - y_{in})$$

Case 2 – if $y = t$ then, $w_i(new) = w_i(old)$

$$b(new) = b(old)$$

Here 'y' is the actual output and 't' is the desired/target output. $(t - y_{in})$ is the computed error.

Step 8 – Test for the stopping condition, which will happen when there is no change in weight or the highest weight change occurred during training is smaller than the specified tolerance.

EXPLAIN MULTIPLE ADAPTIVE LINEAR NEURONS (MADALINE).

Madaline which stands for Multiple Adaptive Linear Neuron, is a network which consists of many Adalines in parallel. It will have a single output unit. Three different training algorithms for MADALINE networks called Rule I, Rule II and Rule III have been suggested, which cannot be learned using backpropagation. The first of these dates back to 1962 and cannot adapt the weights of the hidden-output connection.[10] The second training algorithm improved on Rule I and was described in 1988.[8] The third "Rule" applied to a modified network with sigmoid activations instead of signum; it was later found to be equivalent to backpropagation. The Rule II training algorithm is based on a principle called "minimal disturbance". It proceeds by looping over training examples, then for each example, it:

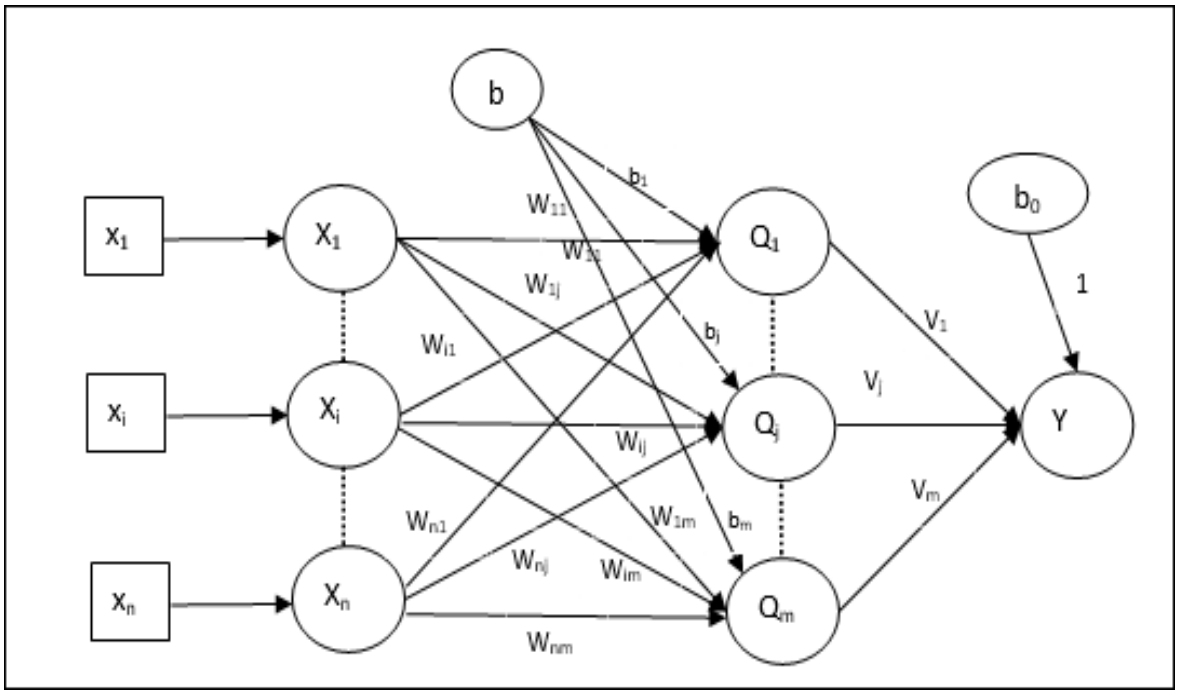
- finds the hidden layer unit (ADALINE classifier) with the lowest confidence in its prediction, tentatively flips the sign of the unit,
- accepts or rejects the change based on whether the network's error is reduced,
- stops when the error is zero.

Some important points about Madaline are as follows:

- It is just like a multilayer perceptron, where Adaline will act as a hidden unit between the input and the Madaline layer.
- The weights and the bias between the input and Adaline layers, as in we see in the Adaline architecture, are adjustable.
- The Adaline and Madaline layers have fixed weights and bias of 1.
- Training can be done with the help of Delta rule.

BRIEFLY EXPLAIN THE ARCHITECTURE OF MADALINE

MADALINE (Many ADALINE) is a three-layer (input, hidden, output), fully connected, feed-forward artificial neural network architecture for classification that uses ADALINE units in its hidden and output layers, i.e. its activation function is the sign function. The three-layer network uses memistors. The architecture of Madaline consists of “n” neurons of the input layer, “m” neurons of the Adaline layer, and 1 neuron of the Madaline layer. The Adaline layer can be considered as the hidden layer as it is between the input layer and the output layer, i.e. the Madaline layer.



Training Algorithm of MADALINE

By now we know that only the weights and bias between the input and the Adaline layer are to be adjusted, and the weights and bias between the Adaline and the Madaline layer are fixed.

Step 1 – Initialize the following to start the training:

- Weights
- Bias
- Learning rate α

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 – Continue step 3-8 when the stopping condition is not true.
Step 3 – Continue step 4-6 for every bipolar training pair s:t.
Step 4 – Activate each input unit as follows:

$$x_i = s_i (I = 1 \text{ to } n)$$

Step 5 – Obtain the net input at each hidden layer, i.e. the Adaline layer with the following relation:

$$O_{inj} = b_j + \sum_{i}^{n} x_i \, w_{ij} \; j = 1 \; to \; m$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output at the Adaline and the Madaline Layer:

$$f(y_{in}) = \begin{cases} 1, if \; x \geq 0 \\ -1, if \; x < 0 \end{cases}$$

Output at the hidden Adaline unit $Q_j = f(Q_{inj})$
Final output of the network $y = f(y_{in})$
i.e.

$$y_{inj} = b_0 + \sum_{j=1}^{m} Q_j \, v_j$$

Step 7 – Calculate the error and adjust the weights as follows –

Case 1 – if y ≠ t and t = 1 then,

$$w_{ij}(new) = w_{ij}(old) + \alpha(1 - Q_{inj}) x_i$$
$$b_j(new) = b_j(old) + \alpha(1 - Q_{inj})$$

In this case, the weights would be updated on Qj where the net input is close to 0 because t = 1.

Case 2 – if y ≠ t and t = -1 then,

$$w_{ik}(new) = w_{ik}(old) + \alpha(-1 - Q_{ink}) x_i$$
$$b_k(new) = b_k(old) + \alpha(-1 - Q_{ink})$$

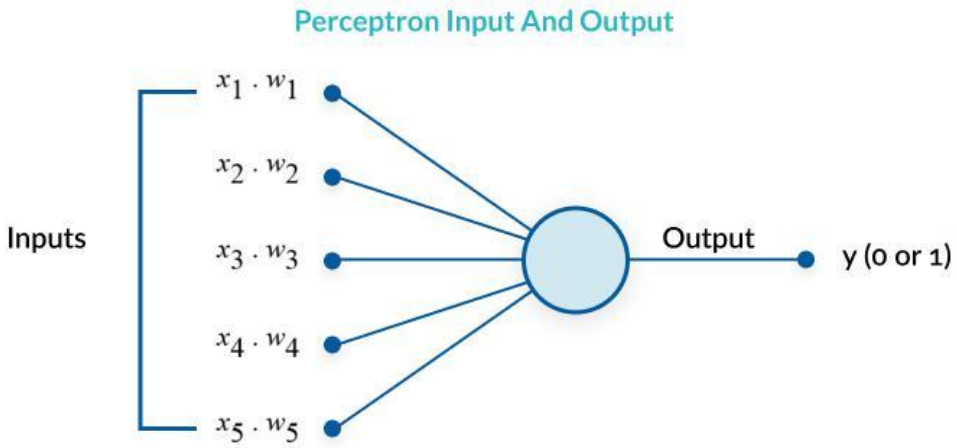
In this case, the weights would be updated on Qk where the net input is positive because t = -1. Here 'y' is the actual output and 't' is the desired/target output.

Case 3 – if y = t, then there would be no change in weights.

Step 8 – Test for the stopping condition, which will happen when there is no change in weight or the highest weight change occurred during training is smaller than the specified tolerance.

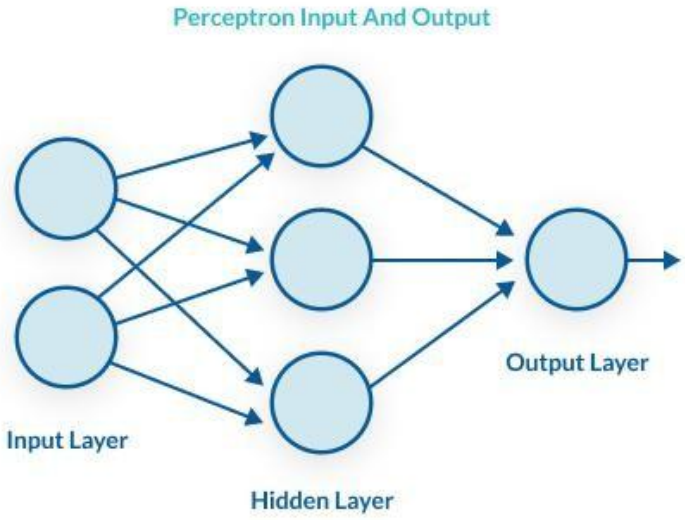
WHAT IS A PERCEPTRON?

A perceptron is a binary classification algorithm modeled after the functioning of the human brain—it was intended to emulate the neuron. The perceptron, while it has a simple structure, has the ability to learn a



What is Multilayer Perceptron?

A multilayer perceptron (MLP) is a group of perceptrons, organized in multiple layers, that can accurately answer complex questions. Each perceptron in the first layer (on the left) sends signals to all the perceptrons in the second layer, and so on. An MLP contains an input layer, at least one hidden layer, and an output layer.



The perceptron learns as follows:

1. Takes the inputs which are fed into the perceptrons in the input layer, multiplies them by their weights, and computes the sum.
2. Adds the number one, multiplied by a "bias weight". This is a technical step that makes it possible to move the output function of each perceptron (the activation function) up, down, left and right on the number graph.
3. Feeds the sum through the activation function—in a simple perceptron system, the activation function is a step function.
4. The result of the step function is the output.

A multilayer perceptron is quite similar to a modern neural network. By adding a few ingredients, the perceptron architecture becomes a full-fledged deep learning system:

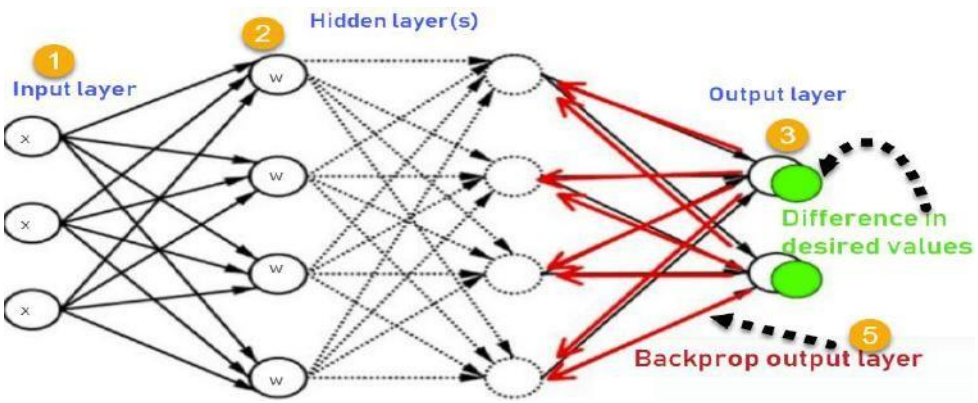
- **Activation functions and other** hyperparameters**:** a full neural network uses a variety of activation functions which output real values, not boolean values like in the classic perceptron. It is more flexible in terms of other details of the learning process, such as the number of training iterations (iterations and epochs), weight initialization schemes, regularization, and so on. All these can be tuned as hyperparameters.
- **Backpropagation**: a full neural network uses the backpropagation algorithm, to perform iterative backward passes which try to find the optimal values of perceptron weights, to generate the most accurate prediction.
- **Advanced architectures**: full neural networks can have a variety of architectures that can help solve specific problems. A few examples are Recurrent Neural Networks (RNN), Convolutional Neural Networks (CNN), and Generative Adversarial Networks (GAN).

WHAT IS BACKPROPAGATION AND WHY IS IT IMPORTANT?

After a neural network is defined with initial weights, and a forward pass is performed to generate the initial prediction, there is an error function which defines how far away the model is from the true prediction. There are many possible algorithms that can minimize the error function—for example, one could do a brute force search to find the weights that generate the smallest error. However, for large neural networks, a training algorithm is needed that is very computationally efficient. Backpropagation is that algorithm—it can discover the optimal weights relatively quickly, even for a network with millions of weights.

HOW BACKPROPAGATION WORKS?

1. **Forward pass**—weights are initialized and inputs from the training set are fed into the network. The forward pass is carried out and the model generates its initial prediction.
2. **Error function**—the error function is computed by checking how far away the prediction is from the known true value.
3. **Backpropagation with gradient descent**—the backpropagation algorithm calculates how much the output values are affected by each of the weights in the model. To do this, it calculates partial derivatives, going back from the error function to a specific neuron and its weight. This provides complete traceability from total errors, back to a specific weight which contributed to that error. The result of backpropagation is a set of weights that minimize the error function.
4. **Weight update**—weights can be updated after every sample in the training set, but this is usually not practical. Typically, a batch of samples is run in one big forward pass, and then backpropagation performed on the aggregate result. The *batch size* and number of batches used in training, called *iterations*, are important hyperparameters that are tuned to get the best results. Running the entire training set through the backpropagation process is called an *epoch*.



Training algorithm of BPNN:

1. Inputs X, arrive through the pre connected path
2. Input is modeled using real weights W. The weights are usually randomly selected.
3. Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.
4. Calculate the error in the outputs

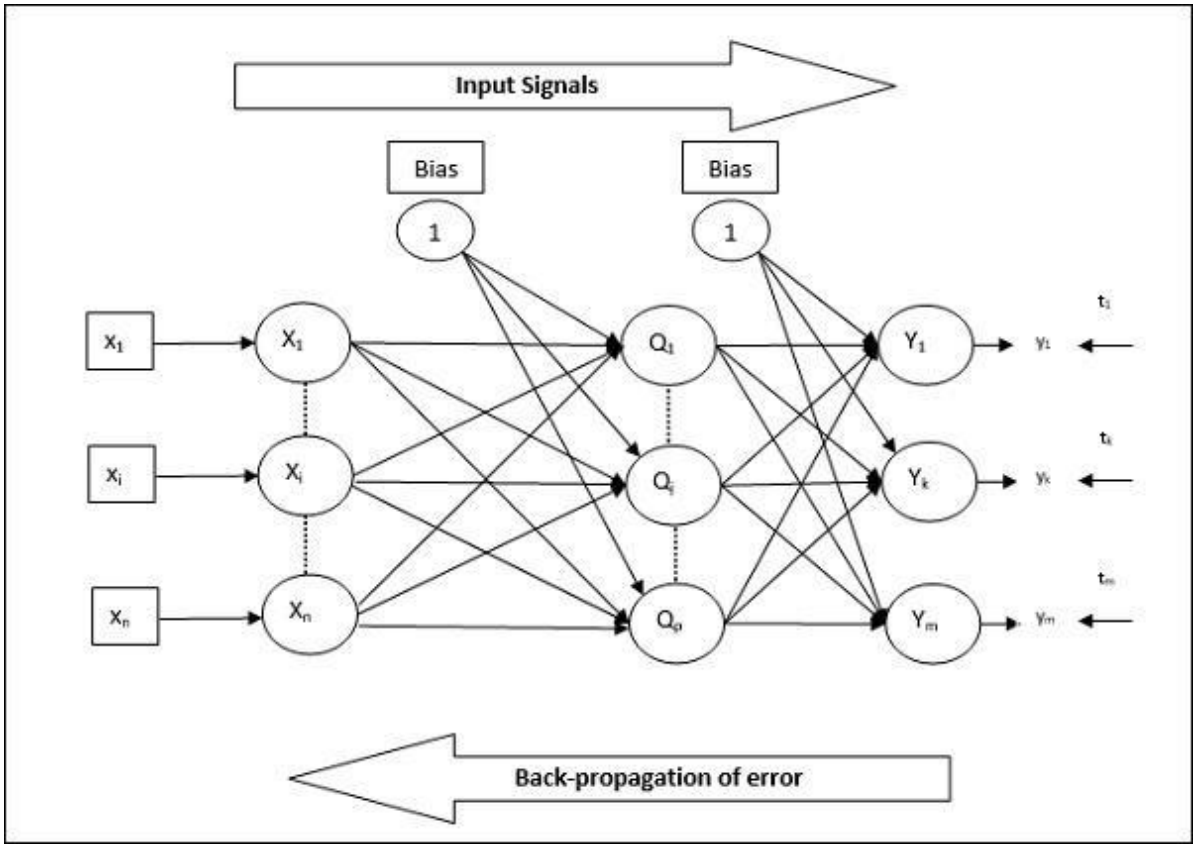
$$Error_B = \text{Actual Output} - \text{Desired Output}$$

5. Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.

Keep repeating the process until the desired output is achieved

Architecture of back propagation network:

As shown in the diagram, the architecture of BPN has three interconnected layers having weights on them. The hidden layer as well as the output layer also has bias, whose weight is always 1, on them. As is clear from the diagram, the working of BPN is in two phases. One phase sends the signal from the input layer to the output layer, and the other phase back propagates the error from the output layer to the input layer.



UNIT-2

WHAT ARE THE ANN LEARNING PARADIGMS?

Learning can refer to either acquiring or enhancing knowledge. As Herbert Simon says, Machine Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task or tasks drawn from the same population more efficiently and more effectively the next time.
ANN learning paradigms can be classified as supervised, unsupervised and reinforcement learning. Supervised learning model assumes the availability of a teacher or supervisor who classifies the training examples into classes and utilizes the information on the class membership of each training instance, whereas, Unsupervised learning model identify the pattern class information heuristically and Reinforcement learning learns through trial and error interactions with its environment (reward/penalty assignment).
Though these models address learning in different ways, learning depends on the space of interconnection neurons. That is, supervised learning learns by adjusting its inter connection weight combinations with the help of error signals where as unsupervised learning uses information associated with a group of neurons and reinforcement learning uses reinforcement function to modify local weight parameters. Thus, learning occurs in an ANN by adjusting the free parameters of the network that are adapted where the ANN is embedded.

BRIEFLY EXPLAIN SUPERVISED LEARNING.

Supervised learning is based on training a data sample from data source with correct classification already assigned. Such techniques are utilized in feed forward or Multi Layer Perceptron (MLP) models. These MLP has three distinctive characteristics:
1. One or more layers of hidden neurons that are not part of the input or output layers of the network that enable the network to learn and solve any complex problems
2. The nonlinearity reflected in the neuronal activity is differentiable and,
3. The interconnection model of the network exhibits a high degree of connectivity These characteristics along with learning through training solve difficult and diverse problems. Learning through training in a supervised ANN model also called as error backpropagation algorithm. The error correction-learning algorithm trains the network based on the input-output samples and finds error signal, which is the difference of the output calculated and the desired output and adjusts the synaptic weights of the neurons that is proportional to the product of the error signal and the input instance of the synaptic weight. Based on this principle, error back propagation learning occurs in two passes:

Forward Pass: Here, input vector is presented to the network. This input signal propagates forward, neuron by neuron through the network and emerges at the output end of the network as output signal:
$y(n) = \varphi(v(n))$, where $v(n)$ is the induced local field of a neuron defined by $v(n) = \Sigma\, w(n)y(n)$.
The output that is calculated at the output layer $o(n)$ is compared with the desired response $d(n)$ and finds the error $e(n)$ for that neuron. The synaptic weights of the network during this pass are remains same.

Backward Pass: The error signal that is originated at the output neuron of that layer is propagated backward through network. This calculates the local gradient for each neuron in each layer and allows the synaptic weights of the network to undergo changes in accordance with the delta rule as:

$$\Delta w(n) = \eta * \delta(n) * y(n).$$

This recursive computation is continued, with forward pass followed by the backward pass for each input pattern till the network is converged. Supervised learning paradigm of an ANN is efficient and finds solutions to several linear and non-linear problems such as classification, plant control, forecasting, prediction, robotics etc.

BRIEFLY EXPLAIN UNSUPERVISED LEARNING.

Self-Organizing neural networks learn using unsupervised learning algorithm to identify hidden patterns in unlabeled input data. This unsupervised refers to the ability to learn and organize information without providing an error signal to evaluate the potential solution. The lack of direction for the learning algorithm in unsupervised learning can sometime be advantageous, since it lets the algorithm to look back for patterns that have not been previously considered. The main characteristics of Self-Organizing Maps (SOM) are:

1. It transforms an incoming signal pattern of arbitrary dimension into one or 2 dimensional map and perform this transformation adaptively
2. The network represents feed forward structure with a single computational layer consisting of neurons arranged in rows and columns.
3. At each stage of representation, each input signal is kept in its proper context and,
4. Neurons dealing with closely related pieces of information are close together and they communicate through synaptic connections.

The computational layer is also called as competitive layer since the neurons in the layer compete with each other to become active. Hence, this learning algorithm is called competitive algorithm. Unsupervised algorithm in SOM works in three phases:

Competition phase: for each input pattern x, presented to the network, inner product with synaptic weight w is calculated and the neurons in the competitive layer finds a discriminant function that induce competition among the neurons and the synaptic weight vector that is close to the input vector in the Euclidean distance is announced as winner in the competition. That neuron is called best matching neuron,

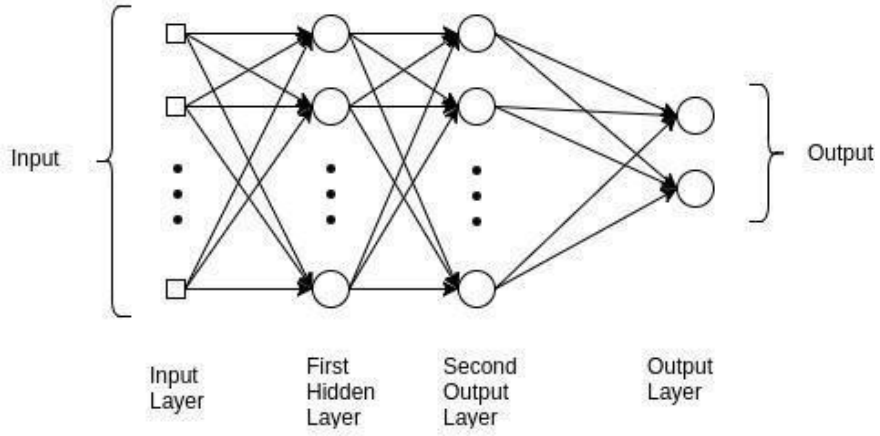
i.e. $x = \arg\min \|x - w\|$.

Cooperative phase: the winning neuron determines the center of a topological neighborhood h of cooperating neurons. This is performed by the lateral interaction d among the cooperative neurons. This topological neighborhood reduces its size over a time period.

Adaptive phase: enables the winning neuron and its neighborhood neurons to increase their individual values of the discriminant function in relation to the input pattern through suitable synaptic weight adjustments, $\Delta w = \eta h(x)(x - w)$. Upon repeated presentation of the training patterns, the synaptic weight vectors tend to follow the distribution of the input patterns due to the neighborhood updating and thus ANN learns without supervisor.

BRIEFLY EXPLAIN MULTI LAYER PERCEPTRON MODEL.

In the Multilayer perceptron, there can more than one linear layer (combinations of neurons). If we take the simple example the three-layer network, first layer will be the input layer and last will be output layer and middle layer will be called hidden layer. We feed our input data into the input layer and take the output from the output layer. We can increase the number of the hidden layer as much as we want, to make the model more complex according to our task.

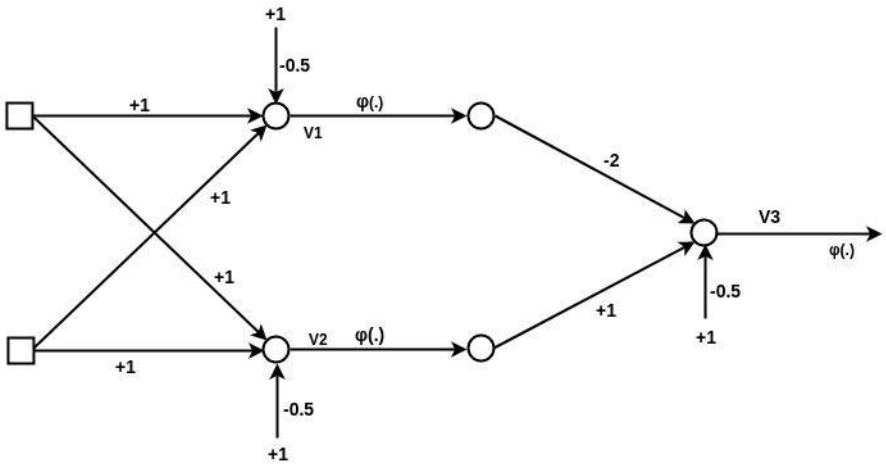


Feed Forward Network, is the most typical neural network model. Its goal is to approximate some function f (). Given, for example, a classifier y = f ∗ (x) that maps an input x to an output class y, the MLP find the best approximation to that classifier by defining a mapping, y = f(x; θ) and learning the best parameters θ for it. The MLP networks are composed of many functions that are chained together. A network with three functions or layers would form f(x) = f (3)(f (2)(f (1)(x))). Each of these layers is composed of units that perform a transformation of a linear sum of inputs. Each layer is represented as y = f(WxT + b). Where f is the activation function, W is the set of parameter, or weights, in the layer, x is the input vector, which can also be the output of the previous layer, b is the bias vector and T is the training function. The layers of an MLP consist of several fully connected layers because each unit in a layer is connected to all the units in the previous layer. In a fully connected layer, the parameters of each unit are independent of the rest of the units in the layer, that means each unit possess a unique set of weights.

Training the Model of MLP: There are basically three steps in the training of the model.

1. Forward pass
2. Calculate error or loss
3. Backward pass

1. Forward pass: In this step of training the model, we just pass the input to model and multiply with weights and add bias at every layer and find the calculated output of the model.



2. Calculate error / loss: When we pass the data instance (or one example) we will get some output from the model that is called Predicted output (pred_out) and we have the label with the data that is real output or expected output(Expect_out). Based upon these both we calculate the loss that we have to backpropagate(using Backpropagation algorithm). There is various Loss Function that we use based on our output and requirement.
3. Backward Pass: After calculating the loss, we back propagate the loss and update the weights of the model by using gradient. This is the main step in the training of the model. In this step, weights will adjust according to the gradient flow in that direction.

Applications of MLP:

1. MLPs are useful in research for their ability to solve problems stochastically, which often allows approximate solutions for extremely complex problems like fitness approximation.
2. MLPs are universal function approximators and they can be used to create mathematical models by regression analysis.
3. MLPs are a popular machine learning solution in diverse fields such as speech recognition, image recognition, and machine translation software.

ELECTRIC LOAD FORECASTING USING ANN

ANNs were first applied to load forecasting in the late 1980's. ANNs have good performance in data classification and function fitting. Some examples of utilizing ANN in power system applications are: Load forecasting, fault classification, power system assessment, real time harmonic evaluation, power factor correction, load scheduling, design of transmission lines, and power system planning. Load forecast has been an attractive research topic for many decades and in many countries all over the world, especially in fast developing countries with higher load growth rate. Load forecast can be generally classified into four categories based on the forecasting time as detailed in the table below.

Load forecasting	Period	Importance
Long term	One year to ten Years	• To calculate and to allocate the required future capacity. • To plan for new power stations to face customer requirements. • Plays an essential role to determine future budget.
Medium term	One week to few months	Fuel allocation and maintenance schedules.
Short term	One hour to a week	• Accurate for power system operation. • To evaluate economic dispatch, hydrothermal co-ordination, unit commitment, transaction. • To analysis system security among other mandatory function.
Very short term	One minute to an hour	Energy management systems (EMS).

An ANN for load forecasting can be trained on a training set of data that consists of time-lagged load data and other non-load parameters such as weather data, time of day, day of week, month, and actual load data. Some ANNs are only trained against days with data similar to the forecast day. Once the network has been trained, it is tested by presenting it with predictor data inputs. The predictor data can be time-lagged load data and forecasted weather data (for the next 24 hours). The forecasted load output from the ANN is compared to the actual load to determine the forecast error. Forecast error is sometimes presented in terms of the root mean square error (RMSE) but

more commonly in terms of the mean absolute percent error (MAPE). An ANN trained on a specific power system's load and weather data will be system dependent. The ANN generated for that system will most likely not perform satisfactorily on another power system with differing characteristics. It is possible the same ANN architecture may be reused on the new system, but retraining will be required.

Training and Testing with ANN

The whole data set was divided into two sets: Training set and Test Set. Training set consists of 80% of whole data and Test set contains the rest data. The training set was used to make a model which, therefore, predicts the load in the future. The model is made by a MATLAB app Neural Net Fitting. The training set has got inputs which are as follows:

1. Temperature (in ^{o} C)
2. Humidity (in %)
3. Pressure (in mBar)
4. Time (in hours)
5. Global Horizontal (in W/m2)
6. Previous Day Same Hour Load (in kW)
7. Previous Week Same Day Same Hour Load (in kW)

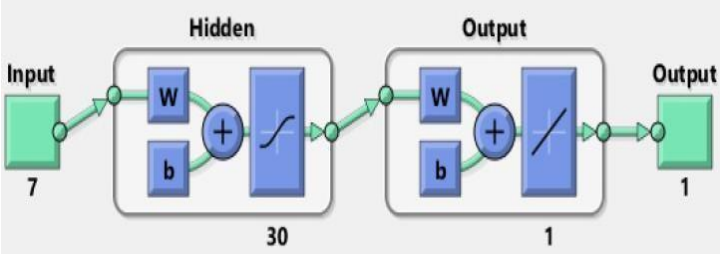
Data Collected: All the following data are collected from substation of feeders:
Maximum, Voltage Minimum, Voltage Maximum, Current Minimum, present MWH consumption, & Temperature.

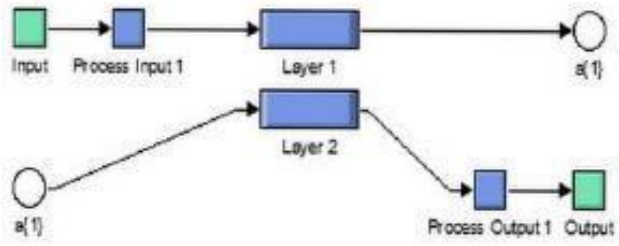
Steps for Implementation of load forecasting using ANN:

- Gathering and arranging the data in MS Excel spreadsheet.
- Tagging the data into groups.
- Analyse the data.
- SIMULINK / MATLAB simulation of data using ANN.

Procedure of testing load forecasting using ANN:

- ANN was created for the user-defined forecast day.
- The data was performed on the training and forecast predictor datasets, the number of hidden layers, or neurons, in the ANN was defined to be 30 neurons.
- The built-in MATLAB Levenberg - Marquardt optimization training function was used to perform the backpropagation training of the feed-forward ANN.
- This process iteratively updated the internal weight and bias values of the ANN to obtain a low error output when utilizing the training predictor dataset and a target dataset.
- The target dataset consists of the actual load values for a given predictor dataset.
- After testing, the ANN forecasted plot was plotted against test set data plot and MAPE was calculated. The results of this forecast were stored, and the entire ANN training, testing, and forecasting process was repeated a set number of times with the intention of reducing the forecast error. The Simulink Model was extracted from the net fitting toolbox is shown below.



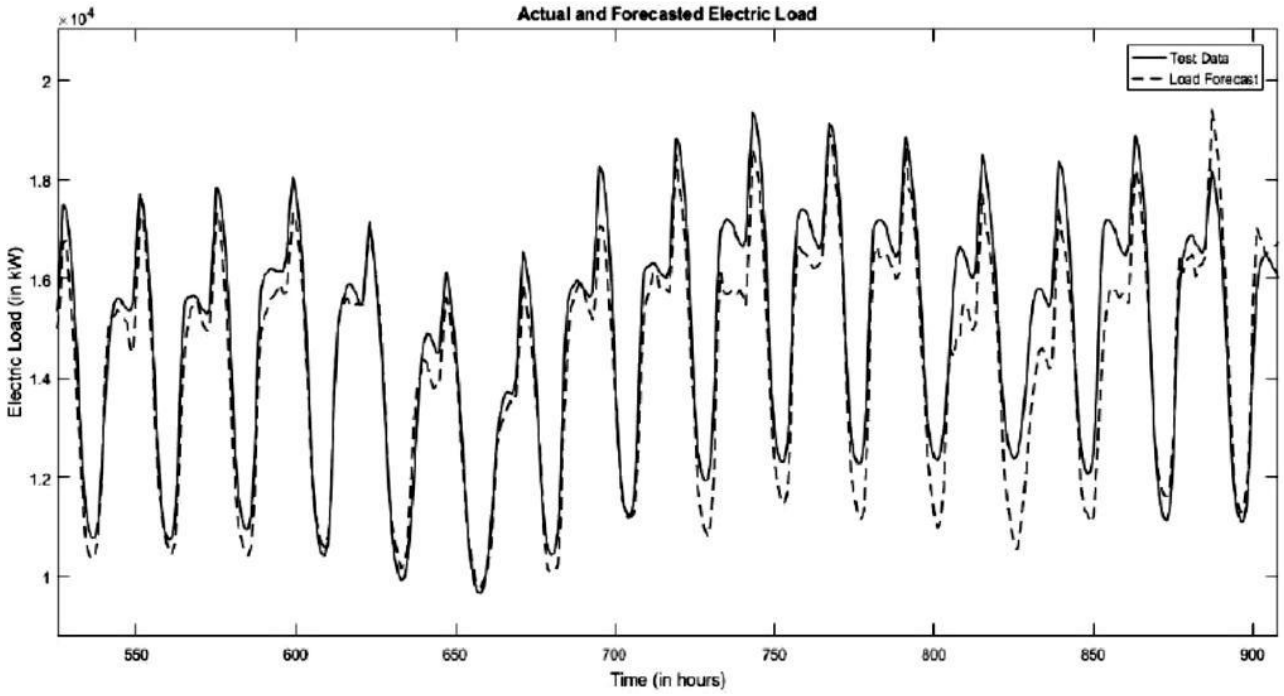


In comparing different models, the average percentage forecasting error is used as a measure of the performance. This is defined as:

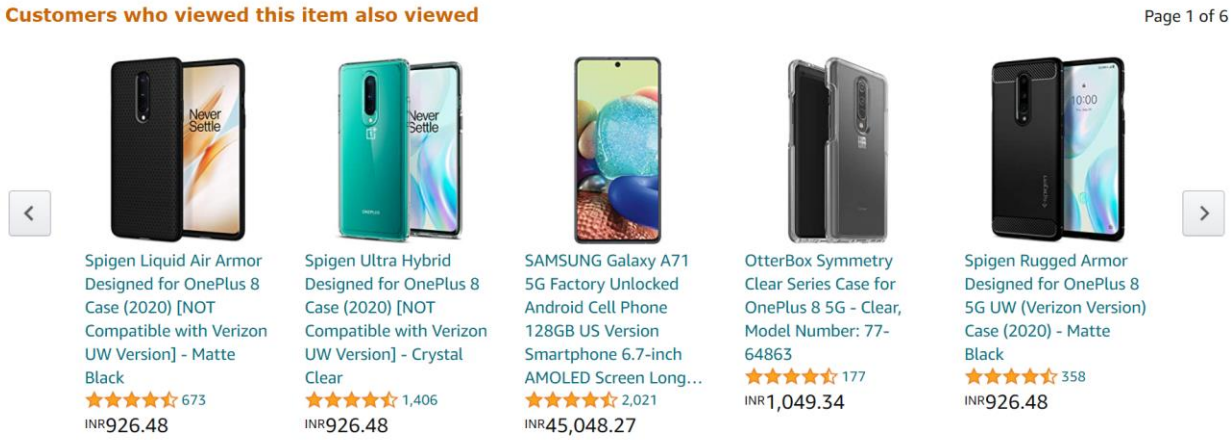
$$E_{mv} = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{\left|\hat{L}_i - L_i\right|}{L_i}\right] \times 100\%$$

where,
N = the number of cases to be forecast (number of hours in
the case of hourly load forecasting).
L_i = the i^{th} load value
$\hat{L}_i$ = the i^{th} load forecast

Result: A graph of forecasted load was plotted against the time (in hours) and comparison was made against the Actual Load (test data load). A part of this graph is shown below. The graph shows a little deviation of the forecasted plot from the Test data load. The MAPE (mean absolute percentage error) came out to be 5.1440 % which is bearable.



RECOMMENDATION SYSTEM



Source: https://www.amazon.com/OnePlus-Glacial-Unlocked-Android-Smartphone/dp/B08723759H/ref=sr_1_3?dchild=1&keywords=mobile&qid=1626099632&sr=8-3

In the above picture of amazon, you may have often seen this page when you try to purchase anything from amazon. These are the recommendation of the product you are trying to purchase and you will be amazed to know that amazon's 35% of the revenue comes from these recommendation engines. Now you may have noticed the power of the recommendation engine. Nowadays every small to big company uses a recommendation engine.

What Is Recommendation System?

A recommendation system is a subclass of Information filtering Systems that seeks to predict the rating or the preference a user might give to an item. In simple words, it is an algorithm that suggests relevant items to users. Eg: In the case of Netflix which movie to watch, In the case of e-commerce which product to buy, or In the case of kindle which book to read, etc.

Use-Cases Of Recommendation System

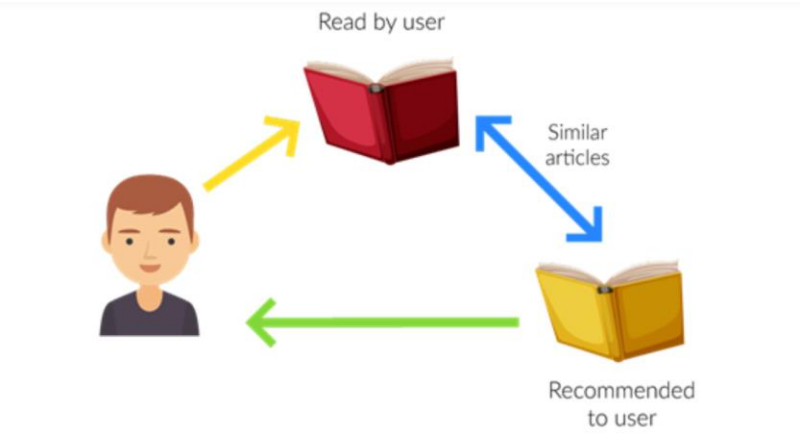
There are many use-cases of it. Some are

A. Personalized Content: Helps to Improve the on-site experience by creating dynamic recommendations for different kinds of audiences like Netflix does.

B. Better Product search experience: Helps to categories the product based on their features. Eg: Material, Season, etc.

TYPES OF RECOMMENDATION SYSTEM

1. Content-Based Filtering



n this type of recommendation system, relevant items are shown using the content of the previously searched items by the users. Here content refers to the attribute/tag of the product that the user like. In this type of system, products are tagged using certain keywords, then the system tries to understand what the user wants and it looks in its database and finally tries to recommend different products that the user wants.

Let us take an example of the movie recommendation system where every movie is associated with its genres which in the above case is referred to as tag/attributes. Now let assume user A comes and initially system don't have any data about user A. so initially, the system tries to recommend the popular movies to the users or the system tries to get some information of the user by getting a form filled by the user. After some time, users might have given a rating to some of the movies like it gives a good rating to movies based on the action genre and a bad rating to the movies based on the anime genre. So here system recommends action movies to the users. But here you can't say that the user dislikes animation movies because maybe the user dislikes that movie due to some other reason like acting or story but actually likes animation movies and needs more data in this case.

Advantage

- Model doesn't need data of other users since recommendations are specific to a single user.
- It makes it easier to scale to a large number of users.
- The model can Capture the specific Interests of the user and can recommend items that very few other users are interested in.

Disadvantage

- Feature representation of items is hand-engineered to some extent, this tech requires a lot of domain knowledge.
- The model can only make recommendations based on the existing interest of a user. In other words, the model has limited ability to expand on the user's existing interests.

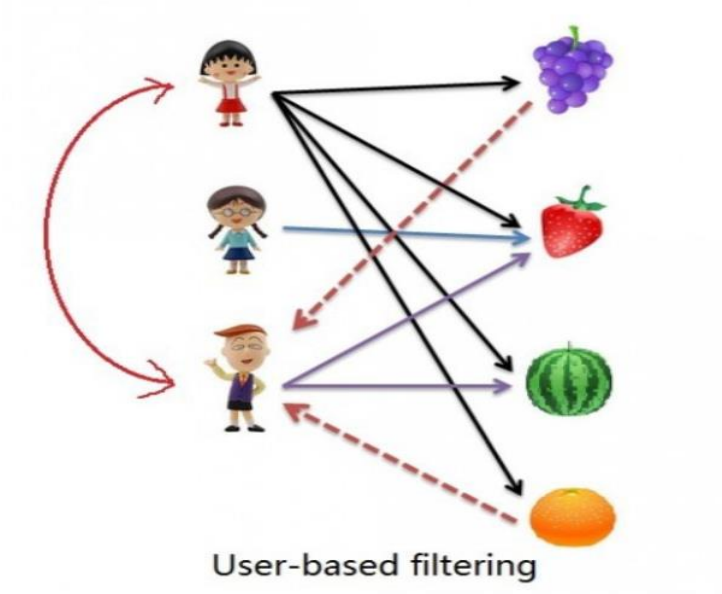
2. Collaborative Based Filtering

Recommending the new items to users based on the interest and preference of other similar users is basically collaborative-based filtering. For eg:- When we shop on Amazon it recommends new products saying *"Customer who brought this also brought"* as shown below.

This overcomes the disadvantage of content-based filtering as it will use the user Interaction instead of content from the items used by the users. For this, it only needs the historical performance of the users. Based on the historical data, with the assumption that user who has agreed in past tends to also agree in future.

There are 2 types of collaborative filtering:-

A. User-Based Collaborative Filtering



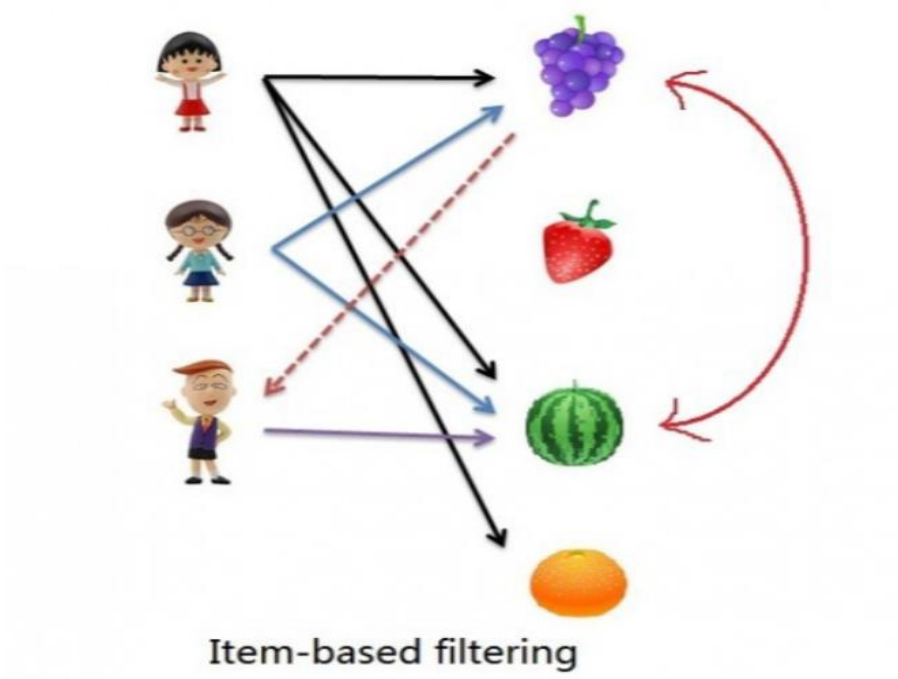
Rating of the item is done using the rating of neighbouring users. In simple words, It is based on the notion of users' similarity.

Let see an example. On the left side, you can see a picture where 3 children named A, B, C, and 4 fruits i.e, grapes, strawberry, watermelon, and orange respectively.

Based on the image let assume A purchased all 4 fruits, B purchased only strawberry and C purchased strawberry as well as watermelon. Here A & C are similar kinds of

users because of this C will be recommended Grapes and Orange as shown in dotted line.

B. Item-Based Collaborative Filtering



The rating of the item is predicted using the user's own rating on neighbouring items. In simple words, it is based on the notion of item similarity.

Let us see with an example as told above about users and items. Here the only difference is that we see similar items, not similar users like if you see grapes and watermelon you will realize that watermelon is purchased by all of them but grapes are purchased by Children A & B. Hence Children C is being recommended grapes.

Now after understanding both of them you may be wondering which to use when. Here is the solution if No. of items is greater than No. of users go with user-based collaborative filtering as it will reduce the computation power and If No. of users is

greater than No. of items go with item-based collaborative filtering. For Example, Amazon has lakhs of items to sell but has billions of customers. Hence Amazon uses item-based collaborative filtering because of less no. of products as compared to its customers.

Advantage

- It works well even if the data is small.
- This model helps the users to discover a new interest in a given item but the model might still recommend it because similar users are interested in that item.
- No need for Domain Knowledge

Disadvantage

- It cannot handle new items because the model doesn't get trained on the newly added items in the database. This problem is known as Cold Start Problem.
- Side Feature Doesn't have much importance. Here Side features can be actor name or releasing year in the context of movie recommendation.

EVALUATION METRICS

As we have discussed different types of recommendation systems their advantages and disadvantages but how can we evaluate whether the given model is recommending the right things or not and how many relevant things this system predicts and here comes evaluation metrics. There are several metrics for evaluating the model but here we will discuss 4 major metrics.

1. Mean Average precision at K

It gives how much relevant is the list of recommended items. Here precision at K means Recommended items in top k sets that are relevant.

2. Coverage

It is the percentage of items in the training data model able to recommend in test sets. Or Simply, the percentage of a possible recommendation system can predict.

3. Personalization

It is basically how many same items the model recommends to different users. Or, the dissimilarity between users lists and recommendations.

4. Intralist Similarity

It is an average cosine similarity of all items in a list of recommendations.

CONTENT BASED RECOMMENDATION SYSTEM

Introduction

One of the most surprising part about Content Based Recommender Systems is, 'we summon to its suggestions / advice every other day, without even realizing that'. Confused? Let me show you some examples. Facebook, YouTube, LinkedIn are among the most used websites on Internet today.

Facebook: Suggests us to make more friends using 'People You May Know' section

Similarly **LinkedIn** suggests you to connect with people you may know and **YouTube** suggests you relevant videos based on your previous browsing history. All of these are recommender systems in action.

While most of the people are aware of these features, only a few know that the algorithms used behind these features are known as 'Recommender Systems'. They 'recommend' personalized content on the basis of user's past / current preference to improve the user experience. Broadly, there are two types of recommendation systems

- **Content Based**
- **Collaborative filtering** based.

CONTENT BASED RECOMMENDATION SYSTEM

- **Item** would refer to content whose attributes are used in the recommender models. These could be movies, documents, book etc.
- **Attribute** refers to the characteristic of an item. A movie tag, words in a document are examples.

What are Content Based Recommender Systems?

Content-based recommender systems are a subset of recommender systems that tailor recommendations to users by analyzing items' intrinsic characteristics and attributes. These systems focus on understanding the content of items and mapping it to users' preferences. By examining features such as genre, keywords, metadata, and other descriptive elements, content-based recommender systems create profiles for both users and items.

This enables the system to make recommendations matching user preferences with items with similar content traits. Unlike collaborative filtering methods that rely on historical interactions among users, content-based systems operate independently, making them particularly useful in scenarios where user history is limited or unavailable. Through this personalized approach, content-based recommender systems play a vital role in enhancing user experiences across various domains, from suggesting movies and articles to guiding users in choosing products or destinations.

How Do Content Based Recommender Systems Work?

A content based recommender works with data that the user provides, either explicitly (rating) or implicitly (clicking on a link). Based on that data, a user profile is generated, which is then used to make suggestions to the user. As the user provides more inputs or takes actions on the recommendations, the engine becomes more and more accurate.

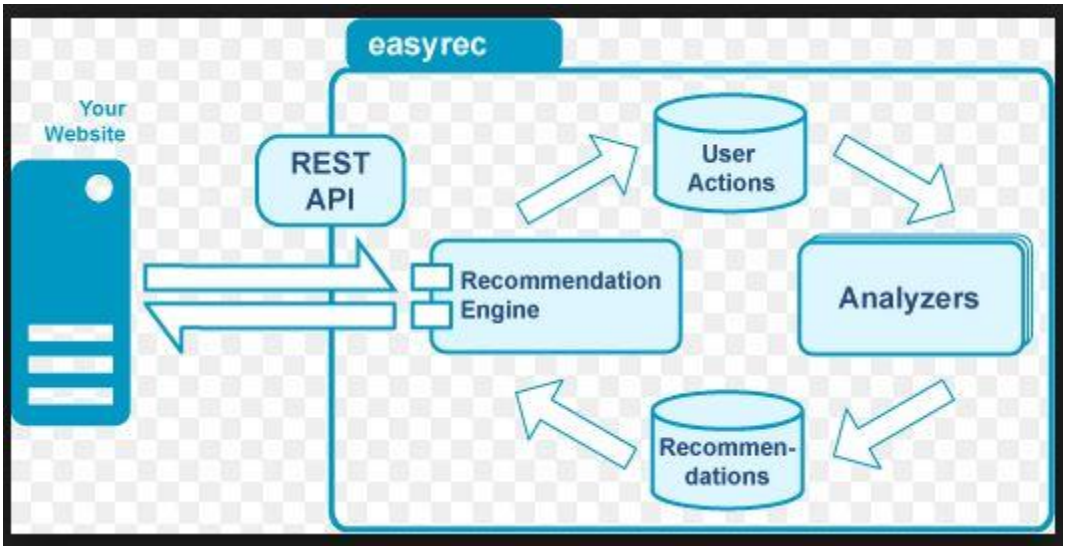


Image Source: easyrec.org

What are the concepts used in Content Based Recommenders?

The concepts of Term Frequency (***TF***) and Inverse Document Frequency (***IDF***) are used in information retrieval systems and also content based filtering mechanisms (such as a content based recommender). They are used to determine the relative importance of a document / article / news item / movie etc.

Term Frequency (TF) and Inverse Document Frequency (IDF)

TF is simply the frequency of a word in a document. IDF is the inverse of the document frequency among the whole corpus of documents. TF-IDF is used mainly

because of two reasons: Suppose we search for ***"the rise of analytics"*** on Google**.** It is certain that "***the"*** will occur more frequently than "***analytics"*** but the relative importance of analytics is higher than the search query point of view. In such cases, TF-IDF weighting negates the effect of high frequency words in determining the importance of an item (document).

But while calculating TF-IDF, log is used to dampen the effect of high frequency words. For example: TF = 3 vs TF = 4 is vastly different from TF = 10 vs TF = 1000. In other words the relevance of a word in a document cannot be measured as a simple raw count and hence the equation below:

Equation:

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d}, & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

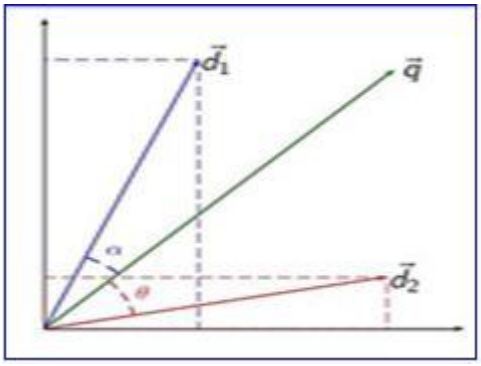
Term Frequency	Weighted Term Frequency
0	0
10	2
1000	4

It can be seen that the effect of high frequency words is dampened and these values are more comparable to each other as opposed to the original raw term frequency.

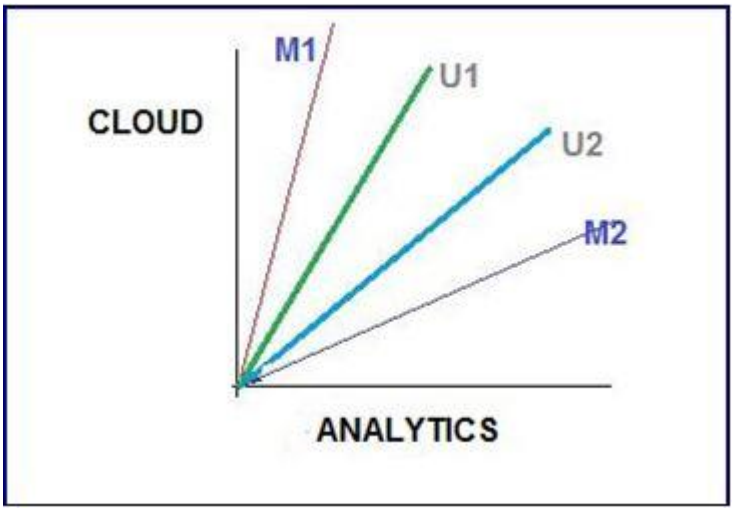
After calculating TF-IDF scores, how do we determine which items are closer to each other, rather closer to the user profile? This is accomplished using the **Vector Space Model** which computes the proximity based on the angle between the vectors.

How does Vector Space Model Works?

In this model, each item is stored as a vector of its attributes (which are also vectors) in an **n-dimensional space** and the angles between the vectors are calculated to **determine the similarity between the vectors**. Next, the user profile vectors are also created based on his actions on previous attributes of items and the similarity between an item and a user is also determined in a similar way.



Lets try to understand this with an example.



Shown above is a 2-D representation of a two attributes, Cloud & Analytics. M1 & M2 are documents. U1 & U2 are users. The document M2 is more about Analytics than cloud whereas M1 is more about cloud than Analytics. I am sure you want to

know how the relative importance of documents are measures. Hold on, we are coming to that.

User U1, likes articles on the topic 'cloud' more than the ones on 'analytics' and vice-versa for user U2. The method of calculating the user's likes / dislikes / measures is calculated by taking the cosine of the angle between the user profile vector***(U_i)*** and the document vector.

The ultimate reason behind using cosine is that the **value of cosine will increase with decreasing value of the angle** between which signifies more similarity. The vectors are length normalized after which they become vectors of length 1 and then the cosine calculation is simply the sum-product of vectors. We will be using the function *sumproduct* in excel which does the same to calculate similarities between vectors in the next section.

Case Study 1: How to Calculate TF – IDF ?

Let's understand this with an example. Suppose, we search for "***IoT and analytics***" on Google and the top 5 links that appear have some frequency count of certain words as shown below:

Articles	Analytics	Data	Cloud	Smart	Insight
Article 1	21	24	***0***	2	2
Article 2	24	59	2	1	***0***
Article 3	40	115	8	10	19
Article 4	4	28	5	***0***	1
Article 5	8	48	4	3	4
Article 6	17	49	8	***0***	5
DF	5,000	50,000	10,000	5,00,000	7000

Among the corpus of documents / blogs which are used to search for the articles, 5000 contains the word ***analytics***, 50,000 contain ***data*** and similar count goes for other words. Let us assume that the total corpus of docs is 1 million(10^6).

Term Frequency (TF)

Articles	Analytics	Data	Cloud	Smart	Insight	Length of Vector
Article 1	2.322219295	2.380211242	0	1.301029996	1.301029996	**3.800456039**
Article 2	2.380211242	2.770852012	1.301029996	1	0	**4.004460697**
Article 3	2.602059991	3.06069784	1.903089987	2	2.278753601	**5.380804488**
Article 4	1.602059991	2.447158031	1.698970004	0	1	**3.527276247**
Article 5	1.903089987	2.681241237	1.602059991	1.477121255	1.602059991	**4.257450611**
Article 6	2.230448921	2.69019608	1.903089987	0	1.698970004	**4.326697114**

As seen in the image above, for article 1, the term Analytics has a TF of **1+ $\log_{10}$21 = 2.322**. In this way, TF is calculated for other attributes of each of the articles. These values make up the attribute vector for each of the articles.

Inverse Document Frequency (IDF)

IDF is calculated by taking the logarithmic inverse of the document frequency among the whole corpus of documents. So, if there are a total of 1 million documents returned by our search query and amongst those documents, '***smart'*** appears in 0.5 million documents. Thus, it's IDF score will be: **Log_{10} (10^6/500000) = 0.30.**

DF	5000	50000	10000	500000	7000
IDF	2.301029996	1.301029996	2	0.301029996	2.15490196
N =	**10^6**				

We can also see (image above), the most commonly occurring term '**smart'** has been assigned the lowest weight by IDF. The length of these vectors are calculated as the **square root of sum of the squared values** of each attribute in the vector:

Articles	Analytics	Data	Cloud	Smart	Insight	Length of Vector
Article 1	2.322219295	2.380211242	0	1.301029996	1.3	=SQRT(J2^2+K2^2+L2^2+M2^2+N2^2)

After we have found out the TF -IDF weights and also vector lengths, let's normalize the vectors.

Articles	Analytics	Data	Cloud	Smart	Insight	Sum of Normalized Lengths
Article 1	0.61103701	0.626296217	0	0.342335231	0.342335231	1
Article 2	0.594389962	0.691941368	0.324895184	0.249721517	0	1
Article 3	0.483581962	0.568817887	0.353681311	0.371691632	0.423496822	1
Article 4	0.454191812	0.693781224	0.481666273	0	0.283504872	1
Article 5	0.447002246	0.62977624	0.376295614	0.34694971	0.376295614	1
Article 6	0.51550845	0.621766675	0.439848211	0	0.392671352	1

Each term vector is divided by the document vector length to get the normalized vector. So, for the term '**Analytics'** in Article 1, the normalized vector is: ***2.322/3.800***. Similarly all the terms in the document are normalized and as you can see (image above) each document vector now has a length of 1.

Now, that we have obtained the normalized vectors, let's calculate the cosine values to find out the similarity between articles. For this purpose, we will take only three articles and three attributes.

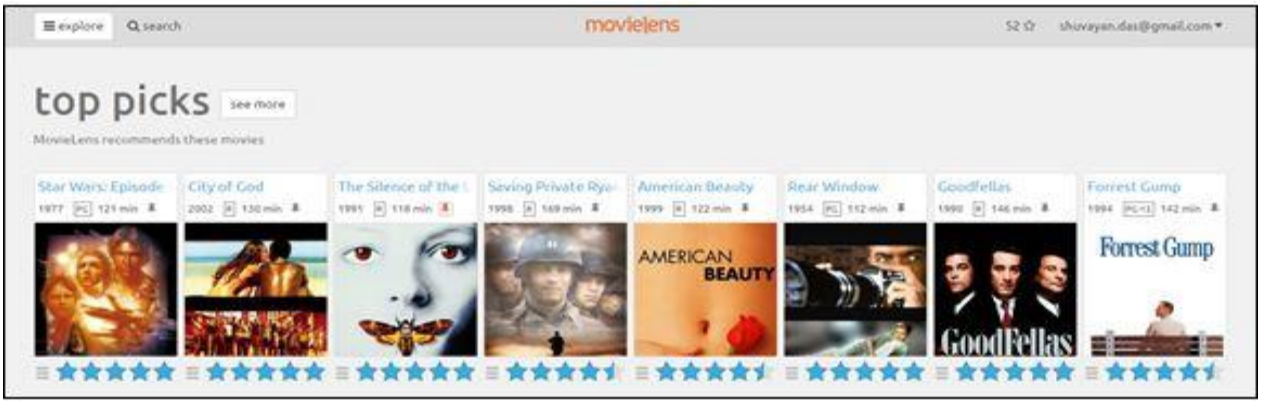
Articles	Analytics	Data	Cloud
Article 1	0.61103701	0.626296217	0
Article 2	0.594389962	0.691941368	0.324895184
Article 3	0.483581962	0.568817887	0.353681311
Article Vector	Cosine Values		
cos(A1,A2)	0.796554526		
cos(A2,A3)	0.795934246		
cos(A1,A3)	0.651734967		

Cos(A1,A2) is simply the sum of dot product of normalized term vectors from both the articles. The calculation is as follows:

Cos(A1,A2) = 0.611*0.59 + 0.63 * 0.69 + 0* 0.32 = =.7965

As you can see the articles 1 and 2 are most similar and hence appear at the top two positions of search results. Also, if you remember we had said that Article 1 (M1) is more about analytics than cloud-Analytics has a weight of 0.611 whereas cloud has 0 after length normalization.

Case Study 2: Creating Binary Representation



This picture is an example of movie recommender system called movielens. This is a **movie recommendation site** which recommends movies you should watch based on which movies you have rated and how much you have rated them.

At a rudimentary level, my inputs are taken in by the system and a profile is created against the attributes which contains my likes and dislikes (*may be based on some keywords / movie tags I have liked,or not done anything about).*

I have liked **52 movies** across various genres. These items (movies) are *attributes* since it can help us to determine a user's taste. Below is the list of top 6 movies recommended by movie-lens. The user columns specify whether a user has

liked / disliked a recommended movie. The 1/0 simply represent whether the movie has adventure/action etc. or not. This type of representation is called a **binary representation of attributes**.

Movie	Adventure	Action	Science-Fiction	Drama	Crime	Thiller		User 1	User 2
Star Wars IV	1	1	1	0	0	0		1	-1
Saving Private Ryan	0	0	0	1	0	0			
American Beauty	0	0	0	1	0	0			
City of Gold	0	0	0	1	1	0		-1	1
Interstellar	0	0	1	1	0	0		1	
The Matrix	1	1	1	0	0	1			1

In the above example, user 1 has liked Star Wars whereas user 2 has not. Based on these types of inputs a user profile is generated.

Other metrics like count data, simple 1 / 0 representation are also used. The measure depends upon the context of use. For example: for **normal movie recommender system,** a simple binary representation is more useful whereas for a search query in search engine like google, a **count representation** might be more appropriate.

Limitations of Content Based Recommender Systems

Content based recommenders have their own limitations. They are not good at capturing inter-dependencies or complex behaviors. For example: I might like articles on Machine Learning, only when they include practical application along with the theory, and not just theory. This type of information cannot be captured by these recommenders.